

FontDoctor: Empowering MLLMs for High-Precision Defect Detection in Chinese Font Libraries

A. ABSTRACT

Chinese font production requires quality control across large character sets, yet manual inspection does not scale. We present FontDoctor, a multimodal large language model (MLLM) for locating and labeling Chinese font defects through two complementary paradigms: few-shot template-referenced detection and non-referenced universal detection. FontDoctor uses SVG XML as an intermediary between raster glyph images and vector outlines, enabling joint reasoning over visual appearance and glyph topology. To alleviate the context bottleneck introduced by multiple templates, a dynamic-resolution, OCR-specialized optical compression encoder transforms variable-size SVG renderings into dense visual embeddings that retain semantic information while reducing sequence length. We further introduce CFDefect-2M, a dataset of two million glyphs with fine-grained defect annotations. Experiments show that FontDoctor achieves state-of-the-art performance and generalizes across font styles in both detection paradigms. These results demonstrate its potential for integration into professional font-quality workflows. Code, model, and data will be released at: <https://fontdoctorgroup.github.io/fontdoctor/>

B. RELATED WORK

B.1. Defect Detection

Defect detection constitutes a pivotal component of industrial production and quality control. Its core objective is to automatically and precisely identify and localize defective regions within images, thereby significantly enhancing product quality and improving the efficiency of automated processing. Mainstream defect detection methodologies can be broadly categorized into two primary streams: traditional rule-based approaches and learning-based approaches.

Edge detection and threshold segmentation employ gradient-based operators to extract edge information, combining global or local thresholding to segregate defect regions from the background, thus determining their locations. Morphological operations utilize techniques such as erosion and dilation to structurally manipulate image shapes, facilitating more accurate separation, denoising, and smoothing of defect areas. Template Matching [1] identifies defect regions by aligning input images with pre-defined defect templates. This process typically necessitates preprocessing steps to eliminate noise and enhance contrast. However, this approach relies strictly on template precision and is incapable of detecting defects with unknown shapes or dimensions. While these methods prove effective in simple, predictable scenarios, their heavy reliance on handcrafted, empirical rules limits their adaptabil-

ity. Consequently, they struggle to accommodate complex, unknown, or diverse defect types and remain highly sensitive to variations in illumination and background texture.

Learning-based methods leverage the analysis of massive labeled or unlabeled training datasets to automatically extract and represent high-level semantic features within images. Convolutional Neural Networks (CNNs), by automatically learning multi-level hierarchical features, effectively perform automated defect detection, demonstrating superior performance particularly when handling complex backgrounds and diverse defects [2]. Two-stage detectors, such as Region Convolutional Neural Networks (R-CNN/Faster R-CNN), utilize bounding box regression [3, 4] to precisely localize defect regions and are capable of identifying multiple defects of varying types and positions. The application of Fully Convolutional Networks (FCNs) in semantic segmentation tasks [5, 6] enables pixel-level classification of images, thereby realizing precise localization and shape delineation of defect regions.

In the context of defect detection, Generative Adversarial Networks (GANs) are utilized not only to synthesize defect images for dataset augmentation but also for defect generation and restoration [7, 8]. By training GANs to reconstruct defect-free (normal) images, regions exhibiting high reconstruction errors serve as the basis for defect identification. This approach is particularly critical in scenarios involving minute defect regions or significant image noise and is frequently applied in unsupervised or semi-supervised detection. Similarly, Autoencoders are often employed for unsupervised defect detection. These models are trained exclusively on normal samples; consequently, they yield poor reconstruction results for defective samples, allowing the reconstruction error to function as an anomaly score. Incorporating the self-attention mechanism, Transformer architectures are capable of capturing global contextual information within images, offering significant advantages in detecting large-scale, irregular, or background-similar defects [9, 10]. However, these methods typically demand extensive annotated data for training, and their robustness and generalization capabilities may be compromised in instances of data imbalance or substantial variations in defect types.

Furthermore, Chinese character defects are not only minute and highly variable but also deeply coupled with font styles. Consequently, semantically agnostic approaches such as coarse-grained bounding box regression or generic segmentation are rendered ineffective, failing to meet the demands for high-precision localization and context-aware assessment.

B.2. Scalable Vector Graphics

Scalable Vector Graphics (SVG) is an XML-based declarative format for graphics, distinguished by its capability to precisely describe the geometric attributes and structural layouts of characters and images. By defining graphic elements via mathematical formulas, SVG ensures resolution independence, maintaining clarity and lossless quality at any scale. This characteristic renders it indispensable in fields such as font design, icon synthesis, and applications requiring high-precision geometric analysis.

In early research on character recognition and graphic digitization, the conversion of raster images into SVG paths, commonly known as image vectorization, predominantly relied on traditional image processing techniques, prioritizing contour extraction and curve fitting. Techniques such as image segmentation [11, 12, 13] are employed to isolate target characters from the background, aiming to derive character contours by extracting edge information. Following contour extraction, polynomial curve fitting [14, 15, 16] (e.g., Bézier curve fitting) is typically utilized to approximate discrete pixels into smooth geometric curves defined by SVG path commands. However, traditional methods are constrained by their inherently bottom-up, local processing nature. They require complex post-processing to optimize geometric precision, struggle to capture the holistic topological structure of characters or graphics, and exhibit high sensitivity to the quality and complexity of input images.

With the rapid advancement of deep learning, SVG generation methods based on sequence modeling architectures have emerged as a prominent direction. By treating SVG path commands, coordinates, and parameters as discrete symbol sequences, these methods successfully bridge the representational gap between vector graphics and raster images. Recurrent Neural Networks (RNNs) [17, 18, 19] and their variant, Long Short-Term Memory (LSTM) networks, have been utilized to model SVG paths as temporal sequences. Representative approaches, such as Im2Vec [20], employ RNNs to encode pixel images into latent representations and subsequently decode them into path sequences. SVG-VAE [21] utilizes class-conditional Variational Autoencoders (VAEs) [22, 23] to learn the latent space of SVG paths. DeepSVG [24] adopts a hierarchical VAE architecture combined with Transformer models to represent and generate complex SVG path structures. The Transformer [25, 26, 27], leveraging its self-attention mechanism, excels at capturing global geometric constraints and structural layouts within SVG sequences, thereby enhancing the coherence and geometric accuracy of generated paths.

In this work, we effectively integrate the task of Chinese font defect detection into the MLLM framework. We first structurally parse Chinese glyphs described by Bézier curves and uniformly encode them into the SVG format. This declarative textual representation not only precisely delineates the vector contour coordinates and topological relationships of the glyphs but also serves as a unified intermediate modality, establishing an effective mapping between the visual features

of raw raster images and fine-grained vector data representations. Consequently, the MLLM is enabled to jointly model and deeply fuse both raster and vector information within a unified framework, thereby realizing a comprehensive and fine-grained analysis and detection of Chinese font defects.

B.3. Multimodal Large Language Models

Multimodal Large Language Models (MLLMs) represent the forefront of the artificial intelligence domain. Built upon the Transformer architecture [28], they have achieved breakthrough advancements in comprehension, generation, and cross-modal reasoning by integrating information from diverse modalities, such as text and vision [29, 30, 31, 32]. The core competencies of MLLMs are derived from the robust foundation of Large Language Models (LLMs). The contextual understanding and logical reasoning capabilities exhibited by LLMs when processing massive volumes of textual data endow them with unprecedented performance across multiple domains; they are capable of comprehending complex natural language queries, conducting semantic analysis, and providing coherent, accurate responses. Furthermore, LLMs possess an inherent advantage in understanding the syntax and semantics of programming languages. Through training on large-scale code datasets (e.g., CodeSearchNet [33], The Pile [34], StarCoder [35]), LLMs have evolved into potent code generators [36, 37, 38], capable of producing high-quality, semantically correct program code, such as Python, SQL, and notably, SVG code. MLLMs synergize the linguistic capabilities of LLMs with advanced visual encoders, achieving deep fusion of cross-modal information via visual-language alignment modules. In tasks such as Image Captioning and Visual Question Answering (VQA), they realize a cognitive leap from merely “seeing” to “understanding and explaining.”

MLLMs also demonstrate robust generalization capabilities and flexibility in tasks like Industrial Anomaly Detection (IAD) [39]. They can utilize prompts to guide the model’s focus toward specific visual anomalies or defect types, analogous to providing verbal instructions to human experts. Fundamentally, SVG is an XML-based code format, and its precise geometric structure aligns perfectly with the code generation and structured understanding capabilities of MLLMs. Consequently, the application of MLLMs in the SVG domain has emerged as a research hotspot, as evidenced by works such as StarVector [40], LLM4SVG [41], and SV-Genius [42]. MLLMs conceptualize image vectorization as an inverse rendering and code generation task. They are capable of generating compact, semantically aligned SVG code directly from images or text descriptions, rather than merely outputting pixel values. Through end-to-end learning strategies, MLLMs can simultaneously model geometric structures and high-level semantic features from large-scale samples [43, 44]. This profound cross-modal understanding capability offers a potential solution to existing technical bottlenecks, holding the promise of achieving precise identification of high-variance, fine-grained defects within challenging

complex character scenarios, thereby overcoming the long-standing limitations of traditional computer vision methods regarding robustness and subjectivity.

C. DATASETS

The existing research lacks publicly available datasets specifically for detecting defects in Chinese characters. To fill this gap and ensure the model’s generalization ability in real-world applications, this study constructs a large-scale, high-quality, and diverse Chinese character defect dataset, CFDefect-2M. The dataset contains 2 million samples, covering various defect patterns from manually designed to deep learning-generated defects.

C.1. Data Construction Pipeline

The data for CFDefect-2M comes from two complementary sources: real design data and model-generated data. This aims to comprehensively cover various defect scenarios from traditional design to intelligent manufacturing.

C.1.1. Real-world Defects from Design Archives.

To construct a high-confidence real defect subset while avoiding the expensive cost of manual annotation, we propose a semi-automated mining strategy based on historical archives, as shown in Fig. 7. We utilized commercial font design project archives accumulated from 2017 to 2025, using commercially released font files that underwent multiple rounds of stringent quality checks as the standard ground truth. These were paired with the original drafts from the design iteration process. By calculating pixel-level differences between the rasterized images, we automatically selected glyphs with significant geometric deviations. These difference areas were then mapped to SVG XML tags and manually reviewed by senior designers. This strategy not only captures real defects resulting from designers’ control over style and details, but also ensures the objectivity of the defect definitions. This portion of the data accounts for approximately 28% (560,000 samples) of the total dataset.

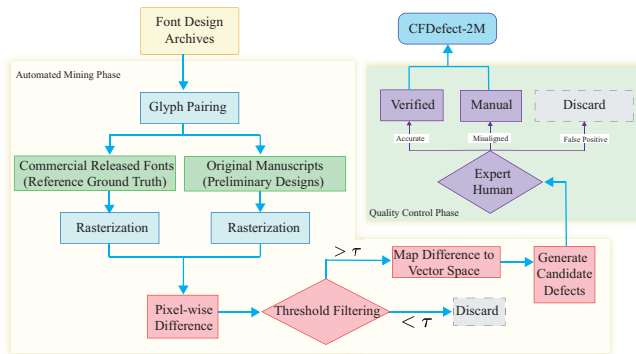


Fig. 7: The semi-automated mining pipeline for real-world defects based on historical design archives.

C.1.2. Synthetic Defects from Generative Models.

As deep learning-based few-shot font generation technology has become a crucial part of industrial font library production, covering the systematic defects produced by such models is particularly important. We constructed a synthetic subset containing 1.44 million samples, integrating seven mainstream SOTA models, including GAN-based **CG-GAN** [45], **FS-Font** [46], **CF-Font** [47], and Diffusion-based **Diff-Font** [48], **FontDiffuser** [49], **MSD-Font** [50], and **FC-Font** [51]. To ensure the accuracy of the data, we designed the following generation pipeline:

1. **Font Image Generation:** We employ the standard Few-shot Generation Paradigm to produce glyph images. Specifically, for each target character, we utilize the standard printed Song typeface as the content reference, while using a small set of randomly sampled target font images as the style reference. These models leverage feature disentanglement and recombination mechanisms to transfer the target style texture onto the source content structure, thereby directly synthesizing rasterized glyph images.
2. **Defect Generation:** Using known standard ground truth, we calculate the differences between the generated glyph and the real glyph at the contour level, and automatically mark the difference areas as initial defect candidates. The marked defects are shown in Fig. 8.
3. **Human Verification:** A manual verification step is introduced to clean up false positives caused by alignment errors or reasonable style variations, retaining only high-confidence systematic defects generated by the model.

C.2. Dataset Organization and Splits

To ensure both the scale of the data and the fairness and diversity of evaluation, CFDefect-2M adopts a differentiated ratio design for the training, validation, and test sets. The specific distribution is shown in Tab. 4.

Table 4: Sample distribution of CFDefect-2M across data sources and dataset splits.

Split	Total Samples	Real Defects	Synthetic Defects	Real Ratio
Training	1,600,000	400,000	1,200,000	25%
Validation	200,000	60,000	140,000	30%
Test	200,000	100,000	100,000	50%
Total	2,000,000	560,000	1,440,000	28%

- **Training Set:** Contains approximately 1.6 million samples, consisting of 200 fonts. Among them, 75% are model-generated samples, and 25% are real defect samples. This ratio aims to fully leverage the scale advantage of automatically generated data, enhancing the model’s robustness to diverse defect patterns.



Fig. 8: Visualization example of the synthetic data automatic defect generation pipeline. By performing contour-level comparisons between model-generated glyphs (Generation) and standard ground truth (Ground Truth), the difference areas are identified and highlighted in red in the “Defect” column.

- **Validation Set:** Contains approximately 200,000 samples, composed of 20 independent fonts. The proportion of real samples is increased to 30%, allowing for more stable performance estimation of real-world scenarios during model selection and hyperparameter tuning.
- **Test Set:** Contains approximately 200,000 samples, composed of another 20 fonts. To eliminate distribution bias and fairly assess model performance, the ratio of real to generated samples in the test set is adjusted to a strict 1:1. Additionally, the test set is further divided into “Seen Characters” (100,000) and “Unseen Characters” (100,000), with a 1:1 ratio of sources within each group. This setup allows us to comprehensively evaluate the FontDoctor model’s overall capability from both the “defect detection in known characters” and “generalization reasoning for unseen characters” perspectives.

D. DATA PROCESSING AND CURATION

To effectively integrate the Chinese character font defect detection task into the MLLM framework, we require an intermediate modality that preserves the glyph topology while being comprehensible to language models. To achieve this, we first perform a structured analysis of the Chinese characters described by Bézier curves, and uniformly encode them into

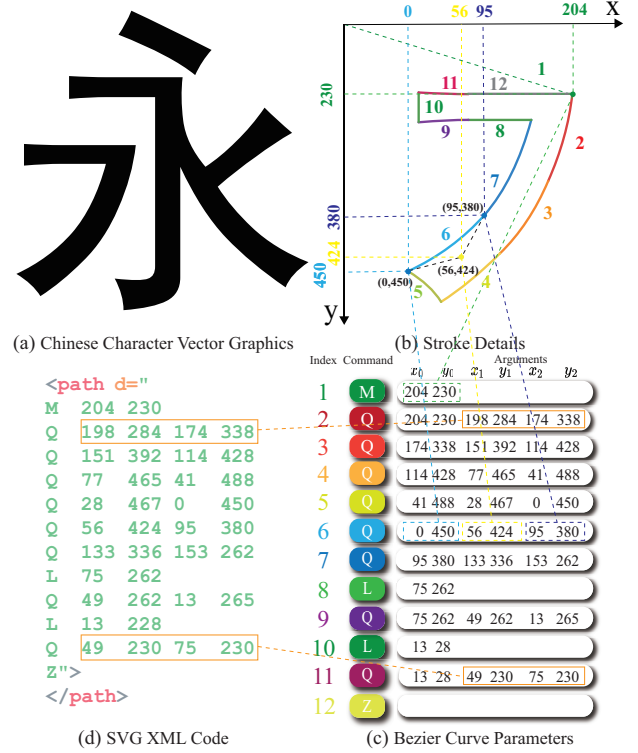


Fig. 9: (a) Vector image of Chinese character fonts. (b) Plotting one stroke on a 2D coordinate axis. (c) Relationship between Bézier curve parameters and the shape. (d) Drawing commands and coordinates represented in XML using absolute coordinates.

Scalable Vector Graphics (SVG) format. In this study, we adopt seven basic drawing commands from the SVG standard to define stroke trajectories (Tab. 5). As shown in Fig. 9(a), we use the Chinese character “永” as an example to illustrate its vector geometric representation mechanism.

Specifically, as shown in Fig. 9(b), we focus on the representative strokes of the character “永” to explain how Bézier curves are used to precisely control the character’s drawing. The drawing process of a character is similar to the movement of a pen tip during human handwriting, controlled by a series of ordered commands that adjust the pen’s state.

As shown in the command table in Fig. 9(c), each row represents a discrete drawing step, with the Index corresponding to different colored segments in Fig. 9 (b). The starting point positioning (M Command): The path first uses the “M” (Move to) command to “lift” the pen and move it to the absolute coordinates (204, 230), establishing the starting position of the stroke (corresponding to Index 1, as shown by the green dashed box). Quadratic Bézier curve drawing (Q Command): The subsequent contours are mainly composed of quadratic Bézier curves. For example, in Index 2, the “Q” command uses a control point (198, 284) to control the curvature of the curve, ultimately reaching the endpoint (174, 338). This parametric curve description allows for the smooth edges of Chi-

nese characters to be perfectly fitted with very few data points. Straight line connection and closure (L & Z Commands): For straight contour segments (e.g., Index 8), we use the “L” (Line to) command to directly connect the coordinate points. Finally, the path is closed using the “Z” (Close path) command, automatically connecting the current point with the starting point, forming a closed geometric region, and completing the contour definition of the stroke.

As shown in Fig. 9(d), the above drawing process is ultimately serialized into a standard XML text description. During this process, we implemented a key coordinate integer discretization optimization strategy. We rounded all floating-point drawing parameters to integers. This strategy is based on two considerations:

Reducing computational overhead: In the Tokenizer of MLLM, floating-point numbers are usually split into multiple tokens, while integers with three digits or fewer are typically treated as a single token. Eliminating the decimal part significantly reduces the total token length of the sequence, thereby improving the model’s inference speed and context window utilization. This is especially critical when processing long sequences with complex strokes.

Glyph consistency: SVG images are typically rendered in screen pixels. On a typical 512×512 or larger canvas, sub-pixel coordinate precision has a negligible effect on the human eye’s ability to detect font defects.

E. SCALABLE VECTOR GRAPHICS

SVG (Scalable Vector Graphics) is an XML-based two-dimensional vector graphic format. Its files consist of several main parts, using the character “永” from the bold font library as an example:

1. XML Declaration. Usually located at the beginning of the file, it declares the XML version and encoding method:

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
```

2. Root Element <svg>. All graphic content is wrapped within the <svg> tag. Common attributes include: xmlns (namespace, typically <http://www.w3.org/2000/svg>), width, height (viewport width and height), and viewBox (defines the visible area in the user coordinate system, formatted as “minX minY width height”).

```
<svg xmlns="http://www.w3.org/2000/svg"
width="512" height="512"
viewBox="0 0 512 512">
<g></g>
</svg>
```

3. Grouping. The <g> tag is used to group multiple elements. Within a <g> group, the class attribute is used to apply uniform styles or behaviors to a group, while the unique id is used to precisely identify and directly access a specific group in styles or scripts.

```
<g class="defect" id="01">
<path></path>
</g>
```

4. Path. In SVG, the <path> element is used to draw complex shapes. The specific shape of the path is defined by the d attribute, which is a string composed of a series of commands and parameters that describe how the path is drawn. Commonly used commands and parameters are shown in Tab. 5.

```
<path d="M 204 230 Q 198 284 174 338 Q 151 392
114 428 Q 77 465 41 488 Q 28 467 0 450 Q 56
424 95 380 Q 133 336 153 262 L 75 262 Q 49 262
13 265 L 13 228 Q 49 230 75 230 Z"></path>
```

F. DEFECT ANNOTATION

Chinese character font defects exhibit high variability and strong subjectivity, encompassing various types such as geometric structure, topological relationships, and style consistency across multiple dimensions. This makes it difficult to accurately generalize and classify them using a predefined set of mutually exclusive category labels. Traditionally, defect detection annotations focus on assigning discrete categories to targets, but in the font domain, the boundaries of many defects are blurred and difficult to strictly define semantically.

To overcome this challenge, this study adopts a precise defect localization annotation method based on vector contours. We fully leverage the advantages of SVG as a declarative graphic representation by directly marking defects on the glyph contours. Specifically, human experts precisely identify and locate the defected stroke segments or path nodes within the defective glyphs, and by overlaying SVG paths, the defect areas are marked in red. As shown in Fig. 10, this marking strategy has the following core advantages:

- **Precise geometric localization:** The vector nature of SVG ensures that defect marking is pixel-independent and highly accurate, capable of precisely covering even the smallest defect areas.
- **Unified modal representation:** By embedding defect information into the SVG text description, we successfully convert visual defect information into structured text features, which are directly used as part of the input sequence for MLLM.
- **Solving category ambiguity:** This localization-based rather than qualitative annotation method avoids the forced and subjective categorization of defects, allowing the model to focus on “where the defect is” and “how severe the defect is,” rather than being constrained by the semantic dilemma of “what type of defect it is.” This greatly enhances the model’s robustness and practical applicability in complex scenarios.

G. DETECTION PARADIGMS

The FontDoctor framework designs two complementary paradigms for Chinese character font defect detection: **Few-Shot Template-Referenced** and **Non-Referenced Universal**. The former performs style-specific defect detection

Table 5: Vector glyphs consist of a different number of SVG drawing commands, with each drawing command representing a parameterized shape contour.

Commands	Arguments	Example	Description
M	x_1, y_1		Move from (x_0, y_0) to (x_1, y_1)
L	x, y		Draw a line from (x_0, y_0) to (x_1, y_1)
H	x_1, y_1		Draw horizontal line from (x_0, y_0) to (x_1, y_0)
V	x_1, y_1		Draw vertical line from (x_0, y_0) to (x_0, y_1)
Q	x_1, y_1 x_2, y_2		Draw quadratic Bézier curve from (x_0, y_0) to (x_2, y_2) , Control point (x_1, y_1)
C	x_1, y_1 x_2, y_2 x_3, y_3 x_4, y_4		Draw cubic Bézier curve from (x_0, y_0) to (x_3, y_3) , Control point (x_1, y_1) and (x_2, y_2)
Z	$\emptyset$		Close path from (x_0, y_0) to (x_0, y_1)

through standard reference templates, while the latter learns a unified defect pattern from global data without explicit template priors.

G.1. Few-Shot Template-Referenced

In the few-shot template-reference scenario, our focus is on how to use a small number of high-quality standard templates to perform flexible defect detection on any target font style. Specifically, given a target font style s , we first select a small number of defect-free standard reference characters from this style to form the template set

$$\mathcal{R}_s = \{(I_{\text{ref}}^{(k)}, S_{\text{ref}}^{(k)})\}_{k=1}^K, \quad (13)$$

where $I_{\text{ref}}^{(k)}$ represents the raster image of the k -th reference character, and $S_{\text{ref}}^{(k)}$ represents the corresponding SVG vector description. For any character to be tested $(I_{\text{qry}}, S_{\text{qry}})$, the model, under the multi-modal large language model (MLLM) framework, takes $\mathcal{R}_s$ and $(I_{\text{qry}}, S_{\text{qry}})$ as joint inputs and encodes them into a unified cross-modal representation. By comparing the differences in geometric structure, stroke topology, and local details between the reference templates and the character to be tested, the model automatically identifies and locates potential defects, ultimately outputting the defect annotations $\mathcal{A}_{\text{qry}}$ in the vector domain. In this task setting, the goal of the model is to learn the mapping relationship from

$$(\mathcal{R}_s, I_{\text{qry}}, S_{\text{qry}}) \longrightarrow \mathcal{A}_{\text{qry}}, \quad (14)$$

where $\mathcal{A}_{\text{qry}}$ uniformly encodes the fine-grained localization of defects in the vector domain for the character to be tested. Unlike traditional methods based on explicit metrics or rigid template matching, the proposed few-shot template-reference paradigm leverages the end-to-end contrastive reasoning capability of MLLM, enabling the model to quickly adapt to new font styles with a small number of template samples, and achieve fine-grained “relative defect” modeling within the same style. The instruction templates used are shown in Tab. 6 #1.

G.2. Non-Referenced Universal

Unlike the few-shot task that relies on style-specific reference templates, the global data non-referenced task aims to learn universal defect patterns and perform intelligent detection solely based on the character to be tested and its global data distribution, without providing any additional style templates. In this setting, we have constructed a training dataset that covers hundreds of font styles and contains a large number of character instances with fine defect annotations:

$$\mathcal{D} = \{(I_i, S_i, \mathcal{A}_i)\}_{i=1}^N, \quad (15)$$

where I_i and S_i represent the raster image and SVG vector representation of the i character, respectively, and $\mathcal{A}_i$ provides its defect spatial localization in the vector domain. Based on the dataset $\mathcal{D}$, we train a detection model:

$$f_{\theta} : (I, S) \rightarrow \mathcal{A}, \quad (16)$$

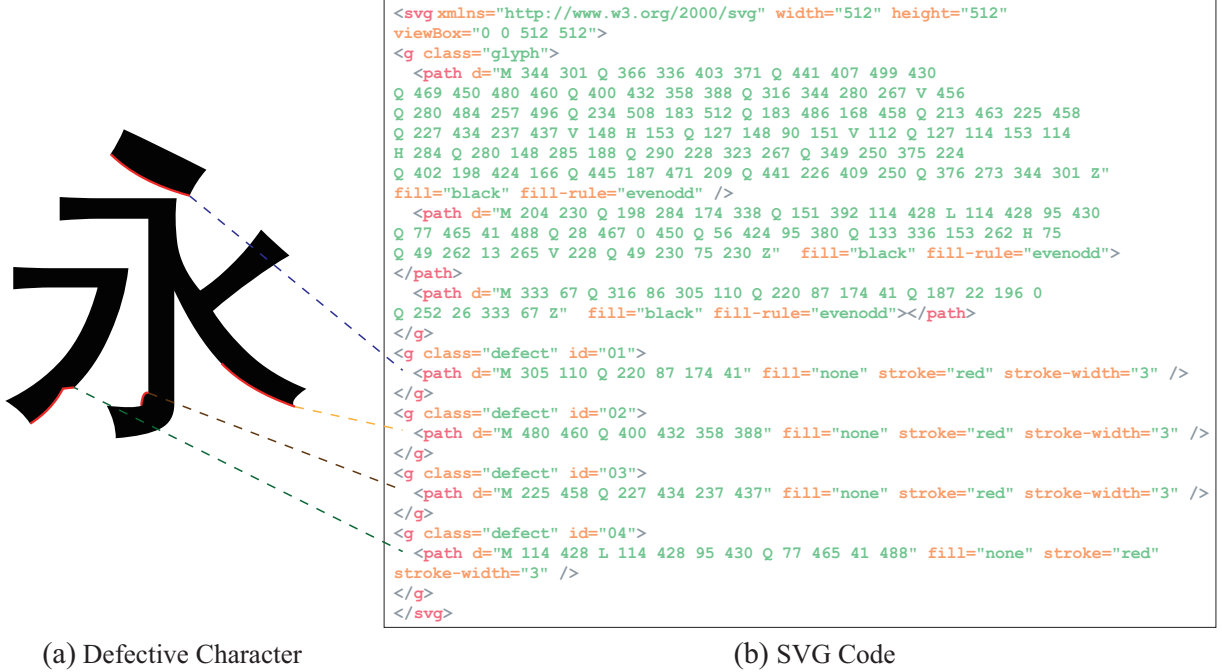


Fig. 10: An illustration of SVG-based defect annotation. (a) A defective character with red overlays. (b) The corresponding SVG code, where each defect region is encoded as an additional `<g class='defect'>` group.

which learns the unified defect pattern across a global distribution of multiple fonts and styles.

In the inference phase, for any character to be tested ($I_{\text{qry}}, S_{\text{qry}}$), the model does not require any style-specific reference templates. Instead, it directly relies on the unified defect pattern learned from $\mathcal{D}$ to determine whether the character has defects, the type of defect, and its spatial location based on global statistical patterns across fonts and styles:

$$f_{\theta}(I_{\text{qry}}, S_{\text{qry}}) = \mathcal{A}_{\text{qry}}, \quad (17)$$

where $\mathcal{A}_{\text{qry}}$ is consistent with the annotation form in the few-shot template-reference task. This paradigm no longer depends on explicit template comparison, but instead, through MLLM’s large-scale pretraining and task-specific fine-tuning, implicitly encodes the common structural rules, typographic norms, and visual aesthetic information across different fonts into a unified representation space.

This global data non-referenced setting better aligns with the needs of large-scale, continuous font quality inspection in real font library production: On one hand, it avoids the cost of building a standard template set for each new style, offering higher automation and deployment efficiency. On the other hand, learning a unified defect pattern enables the model to maintain stable detection performance across multiple fonts and styles, with stronger generalization capabilities for special styles and extreme shape deformations. It is worth emphasizing that the few-shot template-reference and global data non-referenced tasks share the same MLLM framework: the former enhances sensitivity to local subtle differences by introducing style-specific contrastive information, while the

latter strengthens cross-style consistency through large-scale data-driven unified pattern learning. Both complement each other under a unified architecture, forming the core task setting of the FontDoctor framework for Chinese character font defect detection. The instruction templates used are shown in Tab. 6 #2.

H. VISION ENCODER

In the design of the vision encoder architecture, we generally refer to the visual encoder configuration of Qwen2.5-VL, with the specific network structure shown in Tab. 7. We use a 32-layer Vision Transformer (ViT) with a hidden dimension of 1280, 16 attention heads, and an intermediate dimension of 3456 for the feed-forward network. Input images are first divided into 14×14 patches and mapped to 1280-dimensional visual token representations. To avoid quadratic growth in computational complexity with resolution when processing high-resolution or differently sized images, we use global self-attention only at layers 7, 15, 23, 31, and use local window attention with a window size of 112×112 for the other layers. This approach significantly reduces computational overhead while maintaining the ability to model long-range dependencies. For positional encoding, we use 2D Rotary Positional Encoding (2D RoPE) to better capture 2D spatial relationships and employ RMSNorm as the normalization layer and SwiGLU as the activation function.

To improve robustness across different input resolutions, we adopt a dynamic sampling strategy based on the native resolution during training: images are randomly cropped

Table 6: Content templates used for few-shot template-referenced and non-referenced universal detection.

Type	Role	Content Template
# 1	SYSTEM	You are a helpful assistant, based on the given reference images of the font style and their corresponding SVG optical compression images, the SVG XML code is parsed from the SVG optical compression image as the style prior. At the same time, the corresponding content character images and their SVG XML code are obtained as the content branch. The model takes these two parts as joint inputs, performing contrastive learning on a stroke-by-stroke, structure-by-structure basis in a unified feature space. The differences between the two are modeled from multiple dimensions, including geometric shape, topological connectivity, contour smoothness, and detail consistency, to learn fine defects in the content glyph. <code>< /s ></code>
	USER	Style reference image set: <code>{IMG}, ..., {IMG}</code> ; style reference SVG optical compression image set: <code>{SVG}, ..., {SVG}</code> ; glyph image to be detected: <code>{IMG}</code> ; glyph SVG optical compression image to be detected: <code>{SVG}</code> <code>< /s ></code>
	ASSISTANT	<code>SOV Path MoveTo Coord Bézier curve Coord LineTo Coord ... FILL RGB ... EOVS</code> <code>< /s ></code>
# 2	SYSTEM	You are a helpful assistant, without relying on any additional style reference templates, based solely on the given image of the character to be detected and its SVG optical compression image, the corresponding SVG XML code is first parsed and recovered from the SVG optical compression image. The image and SVG XML are then jointly modeled, using the model’s internally learned generic defect patterns to precisely annotate the defect locations in the SVG of the character to be detected. <code>< /s ></code>
	USER	Glyph image to be detected: <code>{IMG}</code> ; glyph SVG optical compression image to be detected: <code>{SVG}</code> <code>< /s ></code>
	ASSISTANT	<code>SOV Path MoveTo Coord Bézier curve Coord LineTo Coord ... FILL RGB ... EOVS</code> <code>< /s ></code>

and scaled according to their original aspect ratio, enabling the model to fully adapt to and generalize across various resolution visual inputs. Additionally, as shown in the Vision-Language Merger section of Tab. 7, we map the 1280-dimensional visual features output by ViT to the 2560-dimensional representation space of the language model through linear projection, thus achieving alignment and fusion of visual tokens and text tokens in the same feature space.

I. VISUAL-SPACE ENCODING OF SYMBOLIC PRIORS

Our proposed SVG Optical Compression strategy is essentially a modality transformation-based semantic compression mechanism. The core of this strategy lies in the fact that OCR-specialized multimodal models already possess outstanding text-image parsing and visual reasoning abilities, enabling them to directly understand the structure and semantics of the code by “reading” the code image, much like humans do. Traditional text Tokenizers, when processing SVG XML, must encode every angle bracket, attribute name, and discrete coordinate value. This character-based serialization introduces significant syntactical redundancy, causing the context window to quickly deplete. In contrast, the process of rendering

Table 7: Network configuration of the Character Vision Encoder.

Configuration	Value
Vision Transformer (ViT)	
Hidden Size	1280
Layers	32
Num Heads	16
Intermediate Size	3456
Patch Size	14×14
Window Size	112×112
Full Attention Block Indexes	{7, 15, 23, 31}
Vision-Language Merger	
In Channel	1280
Out Channel	2560

SVG XML into an image offloads the “syntax parsing” task to the optical compression vision encoder. Specifically, we designed a dynamic rendering function Φ , which simulates a standard code editor view, as shown in algorithm 1.

Optical Compression Vision Encoder. For this visual-space encoding module, FontDoctor uses a separate OCR-

specialized encoder rather than the character vision encoder. We initialize this branch from DeepEncoder, the PyTorch-based visual encoder of DeepSeek-OCR [52], retain only the OCR visual tower, and discard its original text decoder. The rendered SVG XML image is encoded in a dynamic-resolution mode that combines a global high-resolution view with multiple local crops, producing an adaptive sequence of K_{svg} OCR visual tokens. The tokens are projected into the language hidden space through $g_{ocr}(\cdot)$. This branch therefore provides compact code-aware visual embeddings for variable-height SVG code images while leaving the character vision encoder dedicated to raster glyph appearance.

To ensure the clarity of visual features and simulate the best reading experience, we limit the maximum canvas width to $W_{max} = 512$ pixels. If a line of code is shorter (width $w < 512$), the right-side whitespace is cropped to reduce invalid pixel input; if the code is longer, it is automatically wrapped. The canvas height H increases linearly with the number of text lines, ensuring that all SVG descriptions are fully captured.

After rendering, I_{svg} is passed to $\mathcal{E}_{ocr}$ and converted into an adaptive compact set of Visual Embeddings Z_{svg} .

This transformation compresses the original $O(L)$ length of the text sequence into $O(K_{svg})$ visual token numbers ($K_{svg} \ll L$), greatly increasing the information density. This enables FontDoctor to accommodate dozens of reference templates within a single Context Window, unlocking Few-shot reasoning capabilities enhanced by large-scale retrieval.

J. LANGUAGE MODELING

We directly embed the defect annotation coordinates into the SVG XML to connect fine-grained defect detection with MLLM, enabling precise analysis of font defects through text reasoning. Let the SVG XML text with embedded defect annotations be represented as a character sequence $S = \{c_1, c_2, \dots, c_L\}$, where c_i is the i -th character, and L is the sequence length. Given the character image x_v to be tested and the reference template set $\mathcal{R} = \{Z_{svg}^{(1)}, \dots, Z_{svg}^{(N)}\}$, its conditional probability can be modeled as:

$$p(S | x_v, \mathcal{R}) = \prod_{i=1}^L p(c_i | c_{<i}, x_v, \mathcal{R}). \quad (18)$$

The above autoregressive decomposition form allows us to apply the next token cross-entropy generation training objective to the SVG XML text S .

To enhance fine-grained visual understanding and alignment with standard templates, FontDoctor introduces a DeepStack-style multi-layer visual injection mechanism in the language model. Let the visual encoder $\mathcal{E}_v$ be a ViT with L_v layers, from which intermediate features are extracted from the index set $\mathcal{S} = \{s_1, s_2, s_3\}$, formed by three representative layers:

$$F^{(k)} = \mathcal{E}_v^{(s_k)}(x_v), \quad k = 1, 2, 3. \quad (19)$$

Algorithm 1: SVG Optical Compression.

Input: $\mathcal{S} = \{S_1, S_2, \dots, S_N\}$: A set of SVG XML text strings.

$W_{max} = 512$: Maximum width of the rendered image.

$\mathcal{E}_{ocr}$: Pre-trained dynamic-resolution Optical Compression Vision Encoder.

$g_{ocr}(\cdot)$: OCR Vision-Language Projector (MLP).

Output: $\mathcal{Z} = \{Z^{(1)}, \dots, Z^{(N)}\}$: Compressed visual embeddings.

```

1 Function RenderTextToImage ( $S, W_{max}$ ) :
2   Calculate required text width  $w_{text}$  and height
    $h_{text}$  for string  $S$ 
3   if  $w_{text} < W_{max}$  then
4      $W \leftarrow w_{text}$ 
5   else
6      $W \leftarrow W_{max}$ 
7    $H \leftarrow h_{text}$ 
8    $I \leftarrow \mathbf{1}^{H \times W \times 3}$ 
9    $I \leftarrow \text{Rasterize}(I, S, \mathcal{F}_{mono})$ 
10 Function Main () :
11    $\mathcal{Z} \leftarrow \emptyset$ 
12   for  $i \leftarrow 1$  to  $N$  do
13      $I_{svg}^{(i)} \leftarrow \text{RenderTextToImage}(S_i, W_{max})$ 
14      $F_{ocr}^{(i)} \leftarrow \mathcal{E}_{ocr}(I_{svg}^{(i)})$ 
15      $Z^{(i)} \leftarrow g_{ocr}(F_{ocr}^{(i)})$ 
16     Append  $Z^{(i)}$  to  $\mathcal{Z}$ 

```

Using the vision-language merger module $g^{(k)}(\cdot)$, consisting of two MLP layers, the features are downsampled and projected, aligning them to the latent space of the language model, resulting in multi-layer visual token representations:

$$V^{(k)} = g^{(k)}(F^{(k)}) \in \mathbb{R}^{K \times D}, \quad (20)$$

where K is the number of visual tokens, and D is the hidden dimension of the language model (which is consistent with the dimension of $Z_{svg} \in \mathbb{R}^{K \times D}$).

Let the decoder have L_{dec} layers, with the input hidden state at the l -th layer being $H^{(l)} \in \mathbb{R}^{T \times D}$, where T is the length of the joint text and visual sequence, and let $\mathcal{I}_{vis}$ denote the set of positions of the visual tokens in the sequence. We apply residual injection to the visual positions in several shallow decoder layers. Let the layer numbers of these three layers be l_1, l_2, l_3 , corresponding to $k = 1, 2, 3$, respectively, so we have:

$$\tilde{H}_{\mathcal{I}_{vis}}^{(l_k)} = H_{\mathcal{I}_{vis}}^{(l_k)} + V^{(k)}, H^{(l_k+1)} = \mathcal{B}^{(l_k)}(\tilde{H}^{(l_k)}), \quad (21)$$

where $\mathcal{B}^{(l)}(\cdot)$ denotes the l -th Transformer decoding block. This design injects visual information from different layers into the language model multiple times via residual connections, without increasing the sequence length. This forms

Table 8: LLM Network Configuration.

Hyperparameter	Value
Hidden Size	2,560
Intermediate Size	9,728
In Channel	2,560
Out Channel	2,560
Layers	36
Num Heads	32
Full Attention Indexes	0-35
Context Window Size	40,960

multi-layer cross-layer connections from the visual encoder to the language decoder, allowing for a more thorough utilization of multi-level visual representations to improve the quality of font defect detection.

K. TRAINING STRATEGY

With the dual-encoder design of FontDoctor, training must handle two complementary visual sources: the character vision encoder $\mathcal{E}_v$ models pixel-level glyph geometry, while the frozen OCR-oriented encoder $\mathcal{E}_{ocr}$ compresses rendered SVG XML images into compact code-aware visual embeddings. We therefore design a three-stage training protocol. We first calibrate the SVG optical compression branch so that the model can read the visual code representations produced by $\mathcal{E}_{ocr}$. We then train the character vision encoder to align raster glyph appearance with vector topology. Finally, we fine-tune the defect decoder with curriculum learning while jointly optimizing discrete structure generation and continuous coordinate regression. The complete procedure is summarized in Algorithm 3.

K.1. Stage I: Optical SVG Branch Calibration

In the first stage, we do not update the OCR visual tower itself. Instead, with $\mathcal{E}_{ocr}$ frozen, we train the optical projector $g_{ocr}(\cdot)$ and the language-model interface so that the model can recover the original code from rendered SVG XML images. Specifically, the SVG XML text S is converted into an image I_{svg} by the rendering function Φ , encoded by the frozen $\mathcal{E}_{ocr}$, and projected into the language hidden space by $g_{ocr}(\cdot)$:

$$Z_{svg} = g_{ocr}(\mathcal{E}_{ocr}(\Phi(S))). \quad (22)$$

This stage optimizes the optical branch by maximizing $p(S | Z_{svg})$, allowing it to preserve the syntax and semantic information of SVG code without altering the pretrained OCR capability of $\mathcal{E}_{ocr}$.

K.2. Stage II: Raster-to-Vector Alignment

The second stage trains the character visual branch, enabling $\mathcal{E}_v$ to learn the correspondence between raster glyph appearance and SVG vector topology. Given a real raster glyph image x_v , the model is trained to generate its corresponding SVG XML code S with the objective $p(S | \mathcal{E}_v(x_v))$. This stage mainly updates the character vision encoder, vision-language projection modules, and the language model, allowing the model to map pixel distributions to Bézier control points and path structures. The frozen $\mathcal{E}_{ocr}$ is not updated in this stage.

K.3. Stage III: Curriculum-Based Defect Detection

The third stage fine-tunes the model for the core defect detection task. For each training sample $(x_v, S, \mathcal{A})$, the query SVG is first encoded by the frozen optical branch as $Z_{qry} = g_{ocr}(\mathcal{E}_{ocr}(\Phi(S)))$. In the template-referenced mode, reference templates from the same style are encoded by the same branch to form $Z_{\mathcal{R}}$; in the non-referenced mode, the reference context is set to $\emptyset$. The target defect-annotated SVG sequence $S_{tgt} = S \oplus \mathcal{A}$ is converted into a structural skeleton sequence $\tilde{S}_{tgt}$ and a set of continuous coordinates. During training, we jointly optimize the structural token loss $\mathcal{L}_{tok}$ and the coordinate regression loss $\mathcal{L}_{coord}$. Given the wide complexity range of Chinese characters, we still sort the training data by stroke count and use stochastic task mixing to switch between template-referenced and non-referenced modes.

$$\mathcal{L}_{det} = \mathcal{L}_{tok} + \lambda_{coord} \mathcal{L}_{coord}. \quad (23)$$

In each iteration, a Bernoulli variable $\alpha \sim B(\lambda)$ is introduced to dynamically switch the context condition:

- **Referenced Mode** ($\alpha < \lambda$): The model receives the query glyph image x_v , the query SVG optical embedding Z_{qry} , and the same-style reference embeddings $Z_{\mathcal{R}}$. It locates defects by comparing subtle style differences between the query sample and the reference templates.
- **Universal Mode** ($\alpha \geq \lambda$): The reference context is set to $\emptyset$, while the model still receives x_v and Z_{qry} . In this case, it relies on the global design standards internalized during training to independently detect defects in the query character.

This hybrid training mechanism preserves the template-referenced capability while preventing path dependence on reference templates, thereby improving the robustness of FontDoctor across practical deployment scenarios.

L. BASELINES

To comprehensively evaluate the performance of FontDoctor, we compared our proposed method with the current most representative multimodal large language models (MLLMs) and models specifically designed for vector graphics generation.

Algorithm 3: FontDoctor Training Protocol

Input: $\mathcal{D} = \{(x_v, S, \mathcal{A})\}_{i=1}^N$, $\mathcal{E}_v$, frozen $\mathcal{E}_{ocr}$, g_{ocr} , $\mathcal{M}_\theta$, g_ϕ , ϵ , λ , λ_{coord}

Output: Θ^*

```

1  $\Theta_{opt}, \Theta_{char}, \Theta \leftarrow$  active trainable parameter subsets excluding
  frozen  $\mathcal{E}_{ocr}$ 

/* --- Stage I: Optical SVG Branch
   Calibration (SOR) --- */
// Freeze  $\mathcal{E}_{ocr}$ ; train  $g_{ocr}$  and the language
interface
2 while  $|\Delta \mathcal{L}_{SOR}| > \epsilon$  do
3    $\mathcal{B} \leftarrow \{S\}_{b=1}^B \sim \mathcal{D}$ 
4   for  $S \in \mathcal{B}$  do
5      $I_{svg} \leftarrow \Phi(S)$  // Render SVG to Image
6      $Z_{svg} \leftarrow g_{ocr}(\mathcal{E}_{ocr}(I_{svg}))$  //  $\mathcal{E}_{ocr}$  frozen
7      $\mathcal{L}_{SOR} \leftarrow -\log p(S | Z_{svg})$ 
8      $\Theta_{opt} \leftarrow \Theta_{opt} - \eta \nabla_{\Theta_{opt}} \mathcal{L}_{SOR}$ 
9   end
10 end

/* --- Stage II: Character Raster-to-Vector
   Alignment (RVG) --- */
// Train  $\mathcal{E}_v$  to align raster glyphs with
vector topology
11 while  $|\Delta \mathcal{L}_{RVG}| > \epsilon$  do
12    $\mathcal{B} \leftarrow \{(x_v, S)\}_{b=1}^B \sim \mathcal{D}$ 
13   for  $(x_v, S) \in \mathcal{B}$  do
14      $\mathcal{L}_{RVG} \leftarrow -\log p(S | \mathcal{E}_v(x_v))$ 
15      $\Theta_{char} \leftarrow \Theta_{char} - \eta \nabla_{\Theta_{char}} \mathcal{L}_{RVG}$ 
16   end
17 end

/* --- Stage III: Curriculum Defect
   Detection with Continuous Coordinates --- */
18  $\mathcal{D}_{sorted} \leftarrow \text{Sort}(\mathcal{D}, \text{key} = \text{stroke\_count})$ 
19 for  $e \leftarrow 1$  to  $E_{max}$  do
20   foreach  $\mathcal{B} \subset \mathcal{D}_{sorted}$  do
21      $\mathcal{L}_{total} \leftarrow 0$ 
22     for  $(x_v, S, \mathcal{A}) \in \mathcal{B}$  do
23        $S_{tgt} \leftarrow S \oplus \mathcal{A}$ 
24        $(\tilde{S}_{tgt}, \mathcal{C}) \leftarrow \text{ReplaceCoord}(S_{tgt}, \langle coord \rangle)$ 
25        $Z_{qry} \leftarrow g_{ocr}(\mathcal{E}_{ocr}(\Phi(S)))$ 
26        $\alpha \sim \text{Uniform}(0, 1)$ 
27       if  $\alpha < \lambda \wedge \exists \mathcal{R}_{style}$  then
28          $k \sim \text{Uniform}(1, K)$ 
29          $\mathcal{R}_{sub} \leftarrow \{S^{(j)}\}_{j=1}^k \sim \mathcal{R}_{style}$ 
30          $Z_{\mathcal{R}} \leftarrow \{g_{ocr}(\mathcal{E}_{ocr}(\Phi(s))) \mid s \in \mathcal{R}_{sub}\}$ 
31          $\mathcal{L}_{tok} \leftarrow -\log p(\tilde{S}_{tgt} | x_v, Z_{qry}, Z_{\mathcal{R}})$ 
32       else
33          $\mathcal{L}_{tok} \leftarrow -\log p(\tilde{S}_{tgt} | x_v, Z_{qry}, \emptyset)$ 
34       end
35        $\hat{\mathcal{C}} \leftarrow g_\phi(H_{\langle coord \rangle})$ 
36        $\mathcal{L}_{coord} \leftarrow \frac{1}{|\mathcal{C}|} \sum_{c_m \in \mathcal{C}} \|\hat{c}_m - c_m\|_1$ 
37        $\mathcal{L}_{total} \leftarrow \mathcal{L}_{total} + \mathcal{L}_{tok} + \lambda_{coord} \mathcal{L}_{coord}$ 
38     end
39      $\Theta \leftarrow \Theta - \eta \nabla_{\Theta} \mathcal{L}_{total}$ 
40   end
41 end
42 return  $\Theta$ 
```

Since these baseline models are not specifically designed for font defect detection tasks, to ensure fairness and effectiveness in the comparison, we standardized the fine-tuning strategy and input-output formats, adapting them to the CFDefect-2M dataset for full parameter fine-tuning. To apply them to the fine-grained vector font defect detection task, we used the instructions from Tab. 6 to train the model, which predicts the SVG text sequence of defect annotations in an autoregressive manner. The specific comparison models are as follows:

- **StarVector-8B** [40]: This is the current SOTA model in the vector graphics generation field, optimized specifically for SVG code generation. Unlike general-purpose MLLMs, StarVector introduces a specific SVG Tokenizer to handle vector paths. To adapt it to the defect detection task, we fine-tuned it so that its input contains not only images but also the SVG XML text to be tested, and trained it to output the defect annotation SVG XML text.
- **LLaVA-V1.6-7B** [53, 54]: The LLaVA series is a representative work in the field of visual instruction fine-tuning. The V1.6 version introduces a dynamic high-resolution strategy that better retains edge details in font images. We used its original visual encoder to process raster font images and treated the SVG data as pure text for training.
- **Yi-VL-6B** [55]: Based on the powerful Yi language model, Yi-VL demonstrates excellent visual-text alignment capabilities. We evaluate its multimodal understanding ability when combining font visual features with SVG text descriptions.
- **DeepSeek-VL-7B** [56]: This model has significant advantages in handling logical reasoning and visual details. In this experiment, we investigate whether it can understand the complex coordinate transformations and topological logic in SVG code, thereby locating subtle font defects.
- **Qwen2.5-VL-3B** [57]: As an efficient and powerful lightweight version of Qwen2, Qwen2.5-VL introduces a native dynamic resolution mechanism (Naive Dynamic Resolution), performing excellently in OCR and fine-grained visual recognition tasks. We evaluate its ability to understand subtle font defects and generate corresponding SVG descriptions in our experiment, without any special architectural adjustments.
- **Qwen3-VL-4B** [58]: Qwen3-VL-4B is the latest iteration in the Qwen-VL series, significantly enhancing visual reasoning and long-text comprehension abilities. This model continues to support native dynamic resolution input, effectively preserving fine-grained features in images.

Implementation Details for Baselines. To ensure a fair comparison, all baseline models were fine-tuned on the

CFDefect-2M training set. For general-purpose MLLMs, due to the lack of a specialized SVG compression mechanism, we directly used their built-in Tokenizer to tokenize the SVG XML text. To mitigate the long sequence issue, we truncated redundant coordinate precision exceeding the context window limit at the input end. However, this inevitably resulted in some loss of fine-grained information. All models used the same loss function and optimizer settings, and were evaluated on the same test set through rendered image MSE and vector domain DTW/CD metrics.

M. EVALUATION METRICS

Since Chinese character font defect detection is an emerging research field, there is currently a lack of standardized evaluation metrics for fine-grained vector outputs. Traditional defect detection metrics cannot accurately measure the inherent fine granularity, high variability, and the precision required for vector path accuracy in font defects.

To address this research gap, we propose a dual-modal evaluation system designed to comprehensively and accurately measure the performance of MLLMs in defect localization, path sequences, and geometric structures. This system considers both the model’s defect localization ability in the raster domain and the geometric structure and path accuracy in the vector domain.

M.1. Mean Squared Error

Mean Squared Error (MSE)[59] is used as a pixel-level discrepancy measure to assess the overlap between the model’s predicted defect annotation map and the ground truth annotation map in terms of pixel coverage, evaluating the model’s visual localization accuracy. First, we rasterize the model’s predicted SVG defect annotation $\mathcal{A}_{\text{pred}}$ and the ground truth SVG defect annotation $\mathcal{A}_{\text{target}}$ into grayscale images I_{pred} and I_{target} with a fixed resolution. The MSE is calculated as follows:

$$\text{MSE} = \frac{1}{H \cdot W} \sum_{i=1}^H \sum_{j=1}^W (I_{\text{pred}}(i, j) - I_{\text{target}}(i, j))^2. \quad (24)$$

Due to the subtlety of font defects, even minor pixel deviations can impact visual quality. Therefore, MSE provides an intuitive and quantifiable metric for evaluating the model’s effective coverage and localization ability of defect regions in the raster visual layer.

M.2. Dynamic Time Warping

The essence of Chinese character shapes is vector paths formed by Bézier curves, and defect annotations are modifications to these paths. Dynamic Time Warping (DTW)[60] is a sequence matching algorithm used to calculate the global path similarity between two possibly unequal-length coordinate sequences $P = \{p_1, \dots, p_n\}$ and $Q = \{q_1, \dots, q_m\}$. DTW first uses dynamic programming to find the optimal

alignment path $\mathcal{W}^* = \{(i_k, j_k)\}_{k=1}^K$ with the minimum cumulative distance, where K is the length of the aligned path. The cumulative distance $D(n, m)$ is obtained through the following recursive relationship:

$$D(i, j) = d(p_i, q_j) + \min \begin{cases} D(i-1, j) \\ D(i, j-1) \\ D(i-1, j-1) \end{cases}, \quad (25)$$

where $d(p_i, q_j) = \|p_i - q_j\|_2$ is the local distance. To eliminate the impact of differences in path length, we use the average step cost as the final evaluation metric:

$$\text{DTW} = \frac{D(n, m)}{|\mathcal{W}^*|}. \quad (26)$$

DTW evaluates the deviation of the model’s output SVG coordinate sequence from the ground truth annotation sequence in both overall structure and timing. Lower values indicate that the MLLM has a high degree of accuracy in understanding and generating the global structure of the SVG paths.

M.3. Localized Dynamic Time Warping

Localized Dynamic Time Warping (LDTW)[61] is a variant of DTW that introduces a local constraint $\mathcal{C}$ in dynamic programming, limiting the matching search to a narrow region. The computation process of LDTW is similar to that of DTW, but the optimal alignment path $\mathcal{W}_{\mathcal{C}}^*$ must satisfy the local window constraint. We also use the average step cost as the final evaluation metric.

$$\text{LDTW} = \frac{D_{\mathcal{C}}(n, m)}{|\mathcal{W}_{\mathcal{C}}^*|}. \quad (27)$$

LDTW focuses on evaluating local deformations and precision of the path. In font defect detection, defects are often subtle and localized. Compared to global DTW, LDTW is more sensitive to these small, local path differences, allowing for a more accurate measurement of the model’s fine-grained defect area modeling ability.

M.4. Chamfer Distance

Chamfer Distance (CD)[62] is used to measure the geometric similarity between two point sets, A and B . In our task, A and B are the model’s predicted defect contour point set and the ground truth defect contour point set.

$$\text{CD}(A, B) = \frac{1}{|A|} \sum_{a \in A} \min_{b \in B} \|a - b\|_2 + \frac{1}{|B|} \sum_{b \in B} \min_{a \in A} \|b - a\|_2. \quad (28)$$

CD focuses on the accuracy of the final geometric shape, ignoring differences in the path drawing order and parameterization. It is independent of the steps taken by MLLM to generate SVG code and only evaluates whether the final

drawn defect area contour matches the ground truth contour precisely. CD complements DTW. While DTW focuses on the logic and timing of path sequences, CD emphasizes the precision of the final geometric result.

N. QUANTITATIVE EVALUATION

We further break down the experimental results into two core detection paradigms: Few-Shot Template-Referenced and Non-Referenced Universal. The detailed results are shown in Tab. 9. In the Few-Shot Template-Referenced paradigm, a small number of correct samples are provided as references for style-specific contrastive detection. In this setup, FontDoctor fully leverages its cross-modal reasoning and SVG optical compression capabilities, efficiently integrating style priors into the model. The proposed SVG optical compression strategy converts lengthy SVG text into compact visual features. This significantly extends the model’s effective context window, allowing it to receive and deeply integrate large-scale style template information. As a result, the model can perform precise relative defect modeling within the same style.

In the Non-Referenced Universal paradigm, the model must make judgments and annotations solely based on universal font norms and defect patterns learned from a large-scale dataset, without any explicit style templates. FontDoctor employs the DeepStack multi-layer visual injection mechanism, which residualizes multi-level features from the visual encoder—low-level, mid-level, and high-level—into different stages of the LLM decoder. This deep integration mechanism ensures that, at each step of SVG code generation, the LLM can perceive subtle geometric details such as stroke thickness and contour roughness from the original image.

To verify that FontDoctor does not merely learn artifact patterns produced by generative models, we further split the test set into real design defects, synthetic generation defects, and the mixed full test set. Tab. 10 reports the performance on each subset.

The model achieves the lowest errors on synthetic defects, since they usually contain clearer geometric discrepancies and more stable annotation boundaries. Real defects are more challenging because they involve style tolerance, design intent, and subtle subjective judgments, but FontDoctor still maintains high F1 and low vector errors. The stable performance on the mixed test set demonstrates that the model generalizes to both real production defects and generative artifacts.

To further evaluate generalization, we report performance on seen characters, unseen characters, seen fonts, and unseen fonts. The seen-font results are measured on held-out samples from training font styles, while the unseen-font results are measured on the official test font set. Seen/unseen characters are split according to whether the character content appears in the training set.

Tab. 11 shows that unseen characters and unseen fonts introduce moderate performance degradation, but the gap remains small. This indicates that FontDoctor does not rely

on memorizing specific characters or font styles; instead, it learns transferable glyph geometry, SVG topology, and defect patterns. In particular, the model maintains strong performance on unseen fonts, demonstrating robustness to new font styles.

O. QUALITATIVE EVALUATION

To provide a more intuitive performance evaluation, we present a detailed qualitative comparison between FontDoctor and other SOTA models on the CFDefect-2M dataset in Fig. 11. We selected representative samples covering different font styles and defect types. From the visualization results, significant performance differences can be observed. While general visual models like LLaVA-V1.6, Qwen2.5-VL, and Yi-VL can recognize obvious anomalies in the images, when dealing with Chinese characters—objects that require high geometric precision—the defects they generate typically only cover the approximate region of the defect and fail to accurately align with the stroke edges. Although StarVector-8B is specifically designed for SVG generation, it performs inconsistently in defect detection tasks, which require local recognition. In contrast, FontDoctor demonstrates excellent fine-grained defect localization capabilities.

P. HUMAN EVALUATION

Although the quantitative analysis validates the statistical performance of the model, in real industrial scenarios, font quality assessment largely relies on the tacit knowledge and aesthetic judgment of experts. To verify the effectiveness of **FontDoctor** in real industrial workflows, we conducted a rigorous manual evaluation experiment. We randomly selected 10 different font styles from the test set, with 100 samples from each font, totaling 1,000 test cases. The evaluation team consisted of 20 professional font designers with over 3 years of experience. The experiment was conducted using a blind testing mechanism, with all model outputs anonymized. As shown in Tab. 12, designers scored the results based on detailed criteria from three dimensions. To realistically quantify the model’s improvement in productivity, we defined a key efficiency metric: total time spent on single-character quality inspection. This metric not only tracks the model’s inference delay but also covers the entire process of “model inference → designer review → confirmation → final approval.” This reflects the true cost in real industrial scenarios, where designers, faced with low-quality model outputs, must spend additional time on error checking and corrections.

The quantitative manual evaluation results are shown in Tab. 13, where FontDoctor outperforms the compared methods across various perceptual metrics. In terms of Localization Quality, FontDoctor achieved a high score of 4.75, while general-purpose multimodal models often struggle to precisely align with the curves of font strokes, which leads designers to manually adjust the annotation areas, significantly increasing their workload.

Table 9: Quantitative Evaluations. Detailed quantitative evaluations under different detection paradigms.

Method	Few-Shot Template-Referenced				Non-Referenced Universal			
	MSE ↓	DTW ↓	LDTW ↓	CD ↓	MSE ↓	DTW ↓	LDTW ↓	CD ↓
StarVector-8B	0.068	3.1152	3.5913	0.8149	0.083	3.3562	3.8375	0.8653
DeepSeek-VL-7B	0.062	2.8883	3.3874	0.7316	0.076	3.1254	3.6542	0.7812
Qwen2.5-VL-3B	0.053	2.6256	3.1023	0.5977	0.065	2.8410	3.3105	0.6420
OmniSVG-3B	0.052	2.5988	3.0815	0.5788	0.062	2.7654	3.2210	0.6152
InternSVG-8B	0.049	2.4850	2.9510	0.5120	0.059	2.6845	3.1450	0.5714
SVGen-7B	0.047	2.4120	2.8650	0.4850	0.056	2.6120	3.0845	0.5428
Yi-VL-6B	0.045	2.3586	2.8068	0.4612	0.054	2.5631	3.0124	0.5123
LLaVA-V1.6-7B	0.035	1.9833	2.3978	0.3393	0.042	2.1245	2.5641	0.3841
Qwen3-VL-4B	0.020	1.3981	1.6186	0.1312	0.024	1.5421	1.7852	0.1523
FontDoctor-4B (Ours)	0.011	1.0095	1.0825	0.0519	0.016	1.1786	1.2825	0.0812
FontDoctor-8B (Ours)	0.010	0.9850	1.0210	0.0485	0.014	1.1450	1.2240	0.0745

Table 10: Performance on real, synthetic, and mixed defect test subsets.

Test Subset	MSE ↓	DTW ↓	LDTW ↓	CD ↓	F1 ↑
Real Defects	0.013	1.0927	1.1675	0.0600	0.850
Synthetic Defects	0.009	0.9263	0.9975	0.0438	0.906
Mixed Test Set	0.011	1.0095	1.0825	0.0519	0.878

Table 11: Generalization evaluation on seen/unseen characters and seen/unseen fonts.

Evaluation Split	MSE ↓	DTW ↓	LDTW ↓	CD ↓	F1 ↑
Seen Characters	0.010	0.9642	1.0351	0.0478	0.889
Unseen Characters	0.012	1.0587	1.1349	0.0561	0.866
Seen Fonts	0.009	0.9325	1.0068	0.0442	0.901
Unseen Fonts	0.011	1.0095	1.0825	0.0519	0.878

Efficiency analysis (Time/Char) reveals a strong correlation between result usability and workflow efficiency. The average time spent on manual quality inspection was 82.4 seconds. Although the baseline models have fast inference speeds, due to their low usability, designers need to spend a significant amount of time verifying false positives or relocating vague annotations. For example, when using LLaVA-V1.6, the total time spent was as high as 75.2 seconds, which is only about 10% faster than manual inspection, indicating that low-quality assistance can actually become a distraction.

In contrast, FontDoctor reduces the total time for single-character quality inspection to 14.5 seconds, achieving an efficiency improvement of about 5.7 times. In most cases, de-

signers only need to perform a simple confirmation to complete the task. This paradigm shift from rediscovery to direct confirmation is the core advantage of FontDoctor.

Q. ABLATION STUDIES

Q.1. Effectiveness of SVG Optical Compression

To evaluate SVG optical compression, we compare raw XML text encoding with optical compression under 1-shot and 6-shot settings. As shown in Tab. 14, optical compression converts lengthy, sparse SVG XML into compact visual tokens. In the 6-shot setting, it reduces the average token count from approximately 6423 to 1584 while lowering all geometric error metrics.

Our experimental results strongly demonstrate the significant advantage of the optical compression strategy in addressing the MLLM context window limitation. In the 1-Shot paradigm, the model needs to process 2 SVG sequences. Optical compression reduced the average total token count from ≈ 1278 to ≈ 424 , a reduction of about 66.8%. This improvement in efficiency ensures that the model can perform basic template reference reasoning efficiently even in resource-constrained environments. More importantly, in the 6-Shot paradigm, where 7 SVG sequences need to be processed together, the total token count of the original XML text skyrocketed to ≈ 6423 , while the optical compression strategy successfully kept the total token count at ≈ 1584 , a reduction of about 75.3%. This allows FontDoctor to perform large-scale sequence joint inference within a smaller context window, significantly reducing the $O(L^2)$ computational complexity of MLLM when handling long sequences,

Input	两	亲	哲	宾	弛	帆	制	乾	候	恢	崛	推	稠	梅	珊	婚	砥	朊	唠	酥	鯤
StarVector-8B	两	亲	哲	宾	弛	帆	制	乾	候	恢	崛	推	稠	梅	珊	婚	砥	朊	唠	酥	鯤
DeepSeek-VL-7B	两	亲	哲	宾	弛	帆	制	乾	候	恢	崛	推	稠	梅	珊	婚	砥	朊	唠	酥	鯤
Qwen2.5-VL-3B	两	亲	哲	宾	弛	帆	制	乾	候	恢	崛	推	稠	梅	珊	婚	砥	朊	唠	酥	鯤
OmniSVG-3B	两	亲	哲	宾	弛	帆	制	乾	候	恢	崛	推	稠	梅	珊	婚	砥	朊	唠	酥	鯤
InternSVG-8B	两	亲	哲	宾	弛	帆	制	乾	候	恢	崛	推	稠	梅	珊	婚	砥	朊	唠	酥	鯤
SVGen-7B	两	亲	哲	宾	弛	帆	制	乾	候	恢	崛	推	稠	梅	珊	婚	砥	朊	唠	酥	鯤
Yi-VL-6B	两	亲	哲	宾	弛	帆	制	乾	候	恢	崛	推	稠	梅	珊	婚	砥	朊	唠	酥	鯤
LLaVA-V1.6-7B	两	亲	哲	宾	弛	帆	制	乾	候	恢	崛	推	稠	梅	珊	婚	砥	朊	唠	酥	鯤
Qwen3-VL-4B	两	亲	哲	宾	弛	帆	制	乾	候	恢	崛	推	稠	梅	珊	婚	砥	朊	唠	酥	鯤
FontDoctor-4B(Ours)	两	亲	哲	宾	弛	帆	制	乾	候	恢	崛	推	稠	梅	珊	婚	砥	朊	唠	酥	鯤
FontDoctor-8B(Ours)	两	亲	哲	宾	弛	帆	制	乾	候	恢	崛	推	稠	梅	珊	婚	砥	朊	唠	酥	鯤
Target	两	亲	哲	宾	弛	帆	制	乾	候	恢	崛	推	稠	梅	珊	婚	砥	朊	唠	酥	鯤

Fig. 11: Qualitative comparison with state-of-the-art MLLMs on the CFDefect-2M dataset.

making high-precision few-shot reasoning with a large number of reference templates feasible in practice.

In addition to the significant efficiency improvement, optical compression also brought performance gains. In the 1-Shot paradigm, after applying optical compression, all metrics showed improvement. This indicates that, while performing information “compression,” the visual encoder also functions as a powerful “reader,” efficiently capturing and integrating subtle geometric structure differences between the character to be tested and the reference templates, thereby enhancing the model’s defect detection ability. When 6 templates are introduced, the advantages of optical compression are further amplified, and the performance improvement is most significant.

In conclusion, the ablation study results strongly confirm that the SVG optical compression strategy is an indispensable key component of the FontDoctor framework. Through cross-modal encoding, it solves the context window limitation problem caused by complex SVG descriptions, especially in the Few-Shot Template-Referenced paradigm, which requires multiple reference templates, achieving a dual breakthrough in both efficiency and performance. This provides reliable assurance for high-precision intelligent font quality control.

Q.2. Inference Efficiency of SVG Optical Compression

To further verify the practical benefit of SVG optical compression in alleviating the context bottleneck, we compare text-based SVG encoding and optical compression under the same hardware setting. We use a batch size of 1 for single-character defect detection and report the average input token

count, end-to-end inference latency, peak GPU memory, and throughput under both 1-shot and 6-shot template-referenced paradigms. The results are shown in Tab. 15.

The results show that optical compression not only reduces the input token count but also substantially decreases the inference overhead caused by long-context attention. In the 6-shot setting, raw SVG text causes a rapid increase in token count and GPU memory, reducing throughput to 0.31 chars/s. In contrast, optical compression reduces the average token count to about 1584, lowers peak memory from 45.2GB to 20.8GB, and improves throughput to 0.91 chars/s. This demonstrates that SVG optical compression improves not only detection accuracy but also the practical efficiency of multi-template inference.

Q.3. Effectiveness of Separating the Two Vision Encoders

To verify the necessity of separating the character vision encoder from the SVG optical compression vision encoder, we compare four encoder configurations. The first variant uses the character vision encoder $\mathcal{E}_v$ to process both raster glyph images and rendered SVG XML images. The second variant uses the OCR-oriented encoder $\mathcal{E}_{ocr}$ for both input types. The third variant adopts two encoders but allows $\mathcal{E}_{ocr}$ to be updated during training. The final variant is our full setting, where $\mathcal{E}_v$ is dedicated to raster glyph images and the frozen $\mathcal{E}_{ocr}$ is dedicated to SVG optical compression images. All defect detection metrics are reported under the 6-shot template-referenced setting.

The results in Tab. 16 show that sharing the character vision encoder can model glyph contours, but it lacks OCR

Table 12: Detailed scoring criteria for manual evaluation. The evaluators strictly followed the guidelines below to score the model’s performance.

Score	Defect Validity <i>Does the marked area contain a real defect?</i>	Localization Accuracy <i>Does the SVG path fit the defect precisely?</i>	Inspection Utility <i>How much time does this save?</i>
1	Severe hallucination: The model misjudges normal strokes or style features as defects. There is no actual defect present.	Complete misalignment: The generated annotation is far from the defect location, or the shape is entirely irrelevant.	Negative interference: The annotation is misleading and requires extra effort to check and disregard these incorrect annotations.
2	False positive: The model identifies a very subtle irregularity, but it is acceptable within the font tolerance range.	Loose coverage: The annotation covers the defect but includes many unnecessary areas, failing to accurately outline the contour.	Inefficient hint: The indicated area is too large or vague. While the region is identified, further manual search is needed to locate the specific defect.
3	Ambiguous: The detection area is controversial; depending on the strictness, it could be a defect or a style choice.	Rough alignment: The SVG path roughly aligns with the defect but misses corners or fine details.	Basic hint: Alerts the designer to an anomaly, allowing for quick localization of the issue.
4	Correct detection: The model correctly identifies real defects, although there may be slight deviations in the severity classification.	High accuracy: The SVG path closely follows the defect boundary, with only minor deviations at complex intersections.	Clear guidance: The annotation is clear and semantically directed. The designer can accurately understand the issue with a quick glance.
5	Completely valid: The identified area is an indisputable defect that clearly violates design specifications.	Pixel-level alignment: The SVG path perfectly outlines the defect, accurately capturing the erroneous geometric form.	Precise diagnosis: The designer does not need to search for the issue and can directly adjust the highlighted path, maximizing efficiency.

priors for dense text images, leading to a much higher SVG code reading error rate and weaker template-referenced reasoning. Conversely, sharing the OCR-oriented encoder preserves strong code-reading ability, but its low-resolution and text-oriented representation is insufficient for capturing pixel-level contour defects in Chinese glyphs, resulting in inferior localization performance. The dual-encoder design alleviates this task conflict. Moreover, freezing $\mathcal{E}_{\text{OCR}}$ prevents defect detection fine-tuning from degrading its pretrained OCR capability, yielding the best performance in both SVG optical reading and final defect detection. These results demonstrate that the two visual inputs require substantially different feature representations, and separating the encoders is an important design choice for stable and high-precision font defect detection.

To further rule out the possibility that the improvement only comes from increasing model capacity, we compare a larger unified vision encoder with the dual-encoder designs in terms of parameter count and inference overhead. All models are evaluated under the same 6-shot template-referenced setting. The larger unified encoder increases the capacity of a single visual tower to process both raster glyph images and rendered SVG XML images, but does not introduce a task-specific OCR branch.

As shown in Tab. 17, the larger unified encoder improves over the smaller shared encoder, but it remains inferior to the dual-encoder design while introducing more trainable parameters, higher latency, and larger memory consumption. This indicates that glyph geometry modeling and dense SVG code-image reading cannot be solved simply by using a larger shared visual tower; they require different visual priors. Freezing $\mathcal{E}_{\text{OCR}}$ keeps the total parameter count unchanged but reduces trainable parameters, memory, and latency compared with a trainable OCR branch, while achieving the best CD. This confirms that the frozen OCR encoder preserves pretrained code-reading ability and keeps the overall system complexity manageable.

Q.4. Backbone Generality of the FontDoctor Architecture

To further examine whether the effectiveness of FontDoctor comes from the proposed architecture rather than a single high-performing backbone, we conduct a controlled backbone generality study. We select three representative character vision encoders, three OCR-oriented vision encoders, and three language backbones. In each block of Tab. 18, only one module family is changed, while the remaining modules, SVG optical-compression interface, training schedule, and

Table 13: Human evaluation results comparing FontDoctor with baselines. Time/Char denotes the total turnaround time (seconds) required to reach a verified clean state, including model inference and human post-correction/verification.

Method	Detect. Prec. $\uparrow$	Local. Quality $\uparrow$	Usability $\uparrow$	Time/Char (s) $\downarrow$
<i>Human (Manual)</i>	-	-	-	82.4s
LLaVA-V1.6-7B	1.85	1.42	1.55	75.2s
Yi-VL-6B	2.10	1.88	1.92	68.5s
Qwen2.5-VL-3B	2.45	2.15	2.28	58.4s
DeepSeek-VL-7B	2.68	2.30	2.45	52.7s
StarVector-8B	3.15	2.60	2.88	41.3s
OmniSVG-3B	3.22	2.68	2.94	39.5s
InternSVG-8B	3.28	2.74	3.02	38.1s
SVGen-7B	3.35	2.80	3.06	36.8s
Qwen3-VL-4B	3.42	2.85	3.10	35.6s
FontDoctor-4B (Ours)	4.68	4.75	4.82	14.5s
FontDoctor-8B (Ours)	4.80	4.85	4.88	13.2s

Table 14: Ablation study of SVG optical compression strategy with varying template numbers.

Paradigm	SVG encoding method	Avg. Token $\downarrow$	MSE $\downarrow$	DTW $\downarrow$	LDTW $\downarrow$	CD $\downarrow$
1-Shot	Text-based SVG	≈ 1278	0.021	1.4875	1.7012	0.1388
	Optical Compression	≈ 424	0.015	1.1588	1.2593	0.0813
6-Shot	Text-based SVG	≈ 6423	0.015	1.1980	1.3094	0.0895
	Optical Compression	≈ 1584	0.011	1.0095	1.0825	0.0519

Table 15: Inference efficiency comparison between text-based SVG encoding and SVG optical compression.

Paradigm	SVG Encoding	Avg. Tokens $\downarrow$	Latency (ms) $\downarrow$	GPU Memory (GB) $\downarrow$	Throughput (chars/s) $\uparrow$
1-Shot	Text-based SVG	≈ 1278	812	21.6	1.23
	Optical Compression	≈ 424	498	14.3	2.01
6-Shot	Text-based SVG	≈ 6423	3268	45.2	0.31
	Optical Compression	≈ 1584	1094	20.8	0.91

continuous-coordinate decoder are kept identical. All results are reported under the 6-shot template-referenced setting.

The results show that stronger backbones improve absolute performance, but the architectural pattern remains effective across all tested combinations. Replacing the native-resolution character encoder with generic visual encoders causes a clear degradation in geometric metrics, indicating that glyph defect detection benefits from preserving fine contour details. Replacing DeepEncoder with other OCR-oriented encoders mainly affects OCR CER and consequently weakens template-referenced reasoning, but the model still substantially outperforms image-only variants. Similarly, smaller language backbones reduce reasoning and SVG generation accuracy, yet retain the same performance trend. These observations suggest that the gains of FontDoctor are not merely inherited from a particular model family; instead, the combination of separated visual roles, optical SVG compression, and continuous-coordinate decoding provides a stable architecture that remains effective under different

Table 16: Ablation study on separating the character vision encoder and the SVG optical compression vision encoder.

Encoder Design	SVG Optical Reading		Defect Detection			
	CER $\downarrow$	Syntax Validity $\uparrow$	MSE $\downarrow$	DTW $\downarrow$	LDTW $\downarrow$	CD $\downarrow$
Shared $\mathcal{E}_v$ for glyph and SVG	3.62%	91.8%	0.019	1.3264	1.4690	0.1127
Shared $\mathcal{E}_{ocr}$ for glyph and SVG	0.24%	99.5%	0.017	1.2842	1.4186	0.1045
Dual encoders w/ trainable $\mathcal{E}_{ocr}$	0.74%	98.6%	0.013	1.0878	1.1782	0.0646
Dual encoders w/ frozen $\mathcal{E}_{ocr}$	0.11%	99.90%	0.011	1.0095	1.0825	0.0519

Table 17: Complexity comparison between unified and dual-encoder designs.

Encoder Design	Total Params (B) $\downarrow$	Trainable Params (B) $\downarrow$	Latency (ms) $\downarrow$	GPU Mem. (GB) $\downarrow$	CD $\downarrow$
Shared $\mathcal{E}_v$	4.18	4.18	1018	19.4	0.1127
Larger unified encoder	4.62	4.62	1236	24.7	0.0784
Dual encoders w/ trainable $\mathcal{E}_{ocr}$	4.49	4.49	1168	23.9	0.0646
Dual encoders w/ frozen $\mathcal{E}_{ocr}$	4.49	4.21	1094	20.8	0.0519

backbone choices.

Q.5. Effectiveness of Native-Resolution Encoding

To verify the necessity of the native-resolution character vision encoder, we resize input glyph images to fixed resolutions of 224×224 , 336×336 , and 512×512 , and compare them with the native-resolution encoding strategy adopted in FontDoctor. All settings use the same language model, SVG optical compression branch, and training configuration. The results are shown in Tab. 19.

The results show that fixed-resolution resizing consistently degrades defect localization accuracy, especially on vector-level geometric metrics such as DTW, LDTW, and CD. This is because font defects often appear as subtle contour offsets near control points, local artifacts, or narrow stroke deformations, which can be blurred by scaling and down-sampling. In contrast, native-resolution encoding preserves the original local glyph structure and therefore improves control-point-level localization.

Table 18: Backbone generality study under the FontDoctor architecture. We vary the character vision encoder, OCR vision encoder, and language backbone while keeping the dual-encoder optical-compression interface, training protocol, and continuous-coordinate decoder unchanged.

Varied Module	Character Vision Encoder	OCR Vision Encoder	Language Backbone	OCR CER (%) ↓	MSE ↓	DTW ↓	LDTW ↓	CD ↓	F1 ↑
$\mathcal{E}_v$	CLIP-ViT-L/14	DeepEncoder	Qwen3-4B	0.12	0.018	1.2942	1.4215	0.0994	0.806
	SigLIP-SO400M	DeepEncoder	Qwen3-4B	0.11	0.015	1.1766	1.2861	0.0738	0.842
	Native-Res ViT	DeepEncoder	Qwen3-4B	0.11	0.011	1.0095	1.0825	0.0519	0.878
$\mathcal{E}_{ocr}$	Native-Res ViT	PaddleOCR-VL Visual	Qwen3-4B	0.29	0.014	1.1456	1.2468	0.0721	0.851
	Native-Res ViT	Qwen2.5-VL Visual	Qwen3-4B	0.18	0.013	1.0927	1.1933	0.0618	0.863
	Native-Res ViT	DeepEncoder	Qwen3-4B	0.11	0.011	1.0095	1.0825	0.0519	0.878
$\mathcal{M}_\theta$	Native-Res ViT	DeepEncoder	LLaMA-3.2-3B	0.11	0.016	1.2384	1.3567	0.0885	0.821
	Native-Res ViT	DeepEncoder	Qwen2.5-3B	0.11	0.013	1.1176	1.2114	0.0647	0.856
	Native-Res ViT	DeepEncoder	Qwen3-4B	0.11	0.011	1.0095	1.0825	0.0519	0.878

Table 19: Ablation study of the native-resolution character vision encoder.

Input Resolution	MSE ↓	DTW ↓	LDTW ↓	CD ↓
Resize to 224×224	0.024	1.5820	1.7456	0.1664
Resize to 336×336	0.019	1.3827	1.5079	0.1198
Resize to 512×512	0.015	1.1914	1.2950	0.0772
Native Resolution (Ours)	0.011	1.0095	1.0825	0.0519

Q.6. Attention Visualization of the Character Vision Encoder

To intuitively analyze how the character vision encoder perceives defect regions, we conduct an attention visualization experiment. Specifically, we select test glyphs containing subtle local defects, extract the attention responses over high-level visual tokens from the character vision encoder, and up-sample the attention maps back to the original glyph image space.

As shown in Fig. 12, the visual attention responses are clearly concentrated on local anomalous regions, such as broken strokes, contour artifacts, abnormal connections, and deformations around Bézier control points. This visualization shows that the character vision encoder does not merely learn global glyph appearance, but captures local geometric cues that are highly related to font defects, thereby providing more reliable visual evidence for subsequent SVG defect annotation and continuous-coordinate decoding.

Q.7. Ablation of the Three-Stage Training Strategy

Table 20: Ablation study of the three-stage training strategy.

Training Variant	MSE ↓	DTW ↓	LDTW ↓	CD ↓	F1 ↑
w/o Stage I	0.017	1.2874	1.4016	0.1032	0.803
w/o Stage II	0.021	1.4528	1.6254	0.1390	0.758
w/o Curriculum	0.014	1.1422	1.2483	0.0716	0.842
Full Training (Ours)	0.011	1.0095	1.0825	0.0519	0.878

To validate the contribution of each stage in the proposed training protocol, we remove Stage I SVG optical branch calibration, Stage II raster-to-vector alignment pretraining, and the stroke-count curriculum in Stage III, respectively. All variants are finally fine-tuned on the same defect detection data. The results are reported in Tab. 20.

Removing Stage I weakens the model’s ability to read compressed SVG representations, making template information less reliable. Removing Stage II severely disrupts the alignment between raster glyph appearance and SVG topology, resulting in the largest performance drop. Removing curriculum learning still allows convergence, but reduces training stability on complex characters. The full three-stage protocol achieves the best geometric accuracy and detection F1, confirming the necessity of this progressive training design.

Q.8. Stability Across Stroke Counts

To verify the stability of FontDoctor under different character complexities, we group the test samples by Chinese character stroke count from 1 to 32 and report MSE, DTW, CD, and average inference time for each subset. Characters with more strokes usually contain more complex contour connections, more Bézier control points, and longer SVG path sequences. This experiment therefore reflects the robustness of the model as structural complexity increases.

As shown in Fig. 13, all error metrics increase only mildly as the stroke count grows, without any obvious performance collapse. Even for high-stroke-count characters, MSE, DTW, and CD remain within a low range, indicating that the native-resolution character vision encoder, SVG optical compression, and continuous-coordinate decoder can stably handle complex glyph structures. The inference time grows smoothly with stroke count, mainly due to longer path structures and autoregressive decoding sequences, but the overall trend remains controlled. These results further demonstrate the stability of FontDoctor for complex Chinese font defect detection.

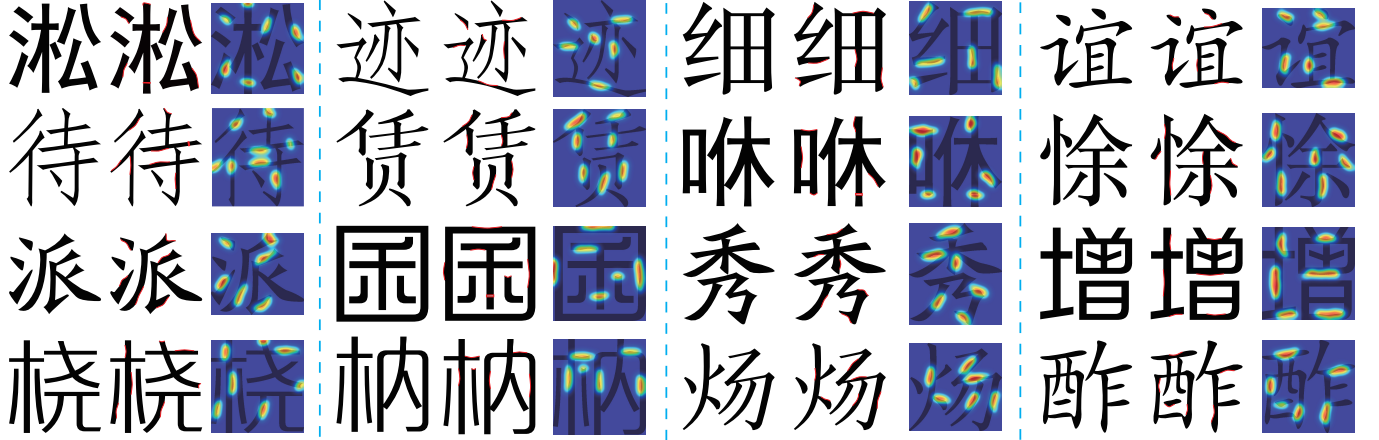


Fig. 12: Attention visualization of the character vision encoder. The attention responses are concentrated on defect-related regions.

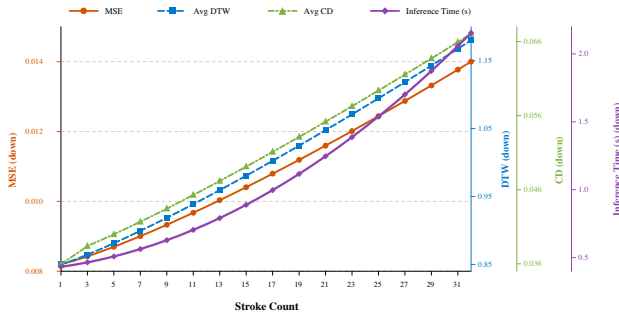


Fig. 13: Defect detection quality and inference time across stroke counts from 1 to 32.

Table 21: Ablation study performance comparison of the FontDoctor framework under different detection paradigms and SVG modalities.

Detection Paradigm	Input Modalities	MSE ↓	DTW ↓	LDTW ↓	CD ↓
Few-Shot Template-Referenced	Image-Only (Baseline)	0.031	1.6253	1.9547	0.2811
	Full Modality (w/ SVG)	0.011	1.0095	1.0825	0.0519
Non-Referenced Universal	Image-Only (Baseline)	0.025	1.5500	1.8010	0.2500
	Full Modality (w/ SVG)	0.016	1.1786	1.2825	0.0812

Q.9. Effectiveness of the SVG Modality

This section systematically evaluates the effectiveness of the SVG modality as a key intermediate representation in the FontDoctor framework and its contribution to the two core detection paradigms through ablation experiments. The essence of Chinese character font defects lies in the geometric or topological anomalies of their vector outlines. Models that rely solely on raster images struggle to achieve fine-grained localization down to curve control points. The experimental results are shown in Tab. 21. Regardless of the detection paradigm, the complete model configuration that introduces

SVG vector descriptions significantly improves detection performance, especially in key metrics that measure vector path differences. This strongly confirms the indispensability of the SVG modality in overcoming the challenges of fine-grained vector structural defect detection.

In the Few-Shot Template-Referenced paradigm, the introduction of the SVG modality leads to a breakthrough in detection accuracy. When the model can simultaneously access the SVG descriptions of both the reference templates and the character to be tested, it gains the ability to perform high-dimensional contrastive reasoning in the precise vector space. This structured vector information eliminates the inherent pixel-level ambiguity of raster images, enabling MLLMs to directly identify and locate subtle errors at the Bézier curve control points or path segments. Compared to the baseline model that relies solely on raster images, the Full Modality configuration greatly enhances the geometric consistency between the output results and the target vector annotations, proving that the SVG modality is a key enabler for achieving contour-level ultra-high precision defect localization.

Q.10. SVG XML vs. Direct Coordinate Representation

To further verify the effectiveness of SVG structural semantics, we compare the SVG XML representation adopted in FontDoctor with direct coordinate representations that do not use SVG. In the Direct Coordinate Sequence baseline, all SVG tags, path commands, attributes, path boundaries, and closure information are removed, leaving only an ordered list of two-dimensional coordinates. In the Command-Coordinate Sequence baseline, simplified command types and coordinates are retained, but the XML hierarchy and attribute semantics are removed. All settings use the same training data, number of reference templates, model size, and decoder configuration; only the vector-side input representation is changed. Path Validity denotes whether the model output can be parsed into a closed and renderable defect path.

Table 22: Ablation study comparing SVG XML with direct coordinate representations.

Vector-side Representation	Command Semantics	Path Validity (%) $\uparrow$	MSE $\downarrow$	DTW $\downarrow$	LDTW $\downarrow$	CD $\downarrow$
Image-Only (w/o vector input)	–	72.4	0.031	1.6253	1.9547	0.2811
Direct Coordinate Sequence (w/o SVG)	No	84.6	0.024	1.4217	1.6075	0.1843
Command-Coordinate Sequence (w/o XML)	Partial	91.2	0.019	1.2546	1.3819	0.1268
SVG XML (Text)	Yes	96.8	0.015	1.1980	1.3094	0.0895
SVG XML + Optical Compression (Ours)	Yes	99.2	0.011	1.0095	1.0825	0.0519

As shown in Tab. 22, the direct coordinate sequence retains numerical positions but removes drawing commands, control-point roles, path grouping, and closure topology. As a result, the model is more likely to confuse starting points, endpoints, and Bézier control points, leading to lower path validity and worse geometric accuracy. Adding simplified commands improves performance, but it remains inferior to full SVG XML, indicating that XML hierarchy, attributes, and path boundaries provide additional structural constraints. The full setting that combines SVG XML with optical compression achieves both the highest path validity and the lowest geometric errors, demonstrating that SVG is not merely a coordinate container, but an effective intermediate modality connecting raster glyphs, vector topology, and language reasoning.

Q.11. Effectiveness of Multi-Layer Visual Injection

To investigate the impact of the depth of visual-language fusion on defect detection performance, we conducted an ablation study on the proposed multi-layer injection mechanism. In the default setting, the visual features extracted from the visual encoder are injected into the first three layers of the LLM decoder. In this experiment, we set the range of injection layers L_{inj} from 0 to 8, with $L_{inj} = 0$ representing no explicit visual feature injection.

As shown in Fig. 14, when $L_{inj} = 3$, the model achieved the best performance across all evaluation metrics. The specific analysis is as follows:

- **Lack of visual guidance ($L_{inj} = 0$):** When visual injection is completely removed, the model performs the worst. This indicates that relying solely on the input token sequence makes it difficult for the LLM to deeply perceive fine-grained geometric features in the raster image, demonstrating the necessity of the cross-layer visual injection mechanism.
- $1 \leq L_{inj} < 3$: When only 1 or 2 layers are injected, the LLM lacks sufficient depth to fully integrate fine visual details with semantic instructions, resulting in higher DTW and CD errors.
- $L_{inj} = 3$: Injecting features into the first three layers allows the model to establish robust initial alignment between the visual structural priors and language

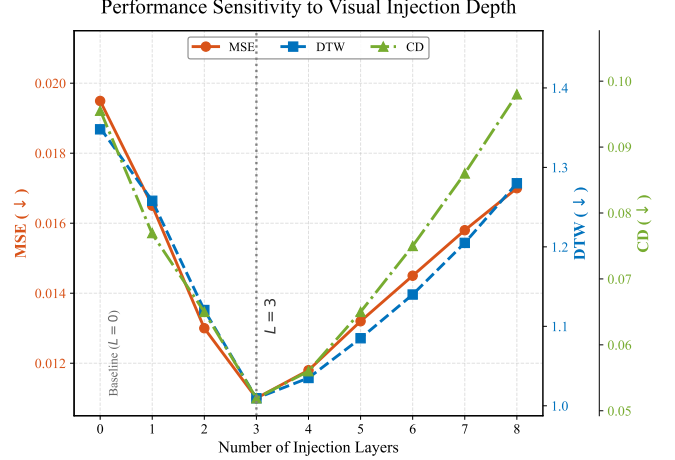


Fig. 14: The impact of visual injection layers on defect detection performance. $L_{inj} = 0$ represents the baseline with no injection. Detection errors are lowest when the injection is set to 3 layers. Excessive injection (> 4 layers) leads to performance degradation due to feature redundancy.

tokens, without negatively interfering with the subsequent language modeling process.

- $L_{inj} > 4$: When the number of injected layers exceeds 4, performance begins to decline. We hypothesize that excessive injection introduces visual noise, disrupting the reasoning flow inside the LLM, leading the model to overfit pixel-level artifacts rather than focusing on high-level topological defects.

To further qualitatively verify whether the injected visual information is actually used by the language decoder, we visualize the feature response maps produced by the multi-layer visual injection mechanism.

As shown in Fig. 15, each case contains the target character content, the query input, two same-style reference glyphs, the corresponding feature maps, and the final SVG defect annotation output. The feature responses are not uniformly distributed over the whole glyph. Instead, they are concentrated around discriminative stroke regions, contour boundaries, and local structures that are important for comparing the query glyph with its style references. This indicates that the residual injection at visual-token positions $\mathcal{L}_{vis}$ preserves

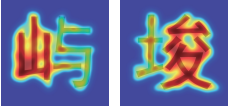
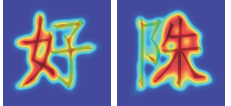
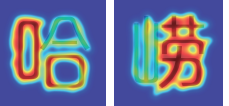
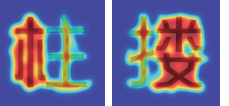
Content	峻	悟	姝	唠	楼
Input	峻	悟	姝	唠	楼
Style Reference	屿 竣	情 晤	好 陈	哈 唠	柱 楼
Feature Map					
Output	峻	悟	姝	唠	楼

Fig. 15: Visualization of the multi-layer visual injection mechanism. For each example, we show the character content, the query input, the style references, the feature response maps after visual injection, and the final defect annotation output. The activated regions in the feature maps are concentrated on style-dependent strokes and local defect-related structures, indicating that the injected visual representations provide effective geometric evidence for SVG defect localization.

style-aware and defect-sensitive cues inside the shallow LLM decoder layers.

The output row further shows that these activated regions are consistent with the final red defect annotations. When the query glyph deviates from the stroke shape implied by the reference style, the feature maps highlight the corresponding local stroke components, and the decoder subsequently generates SVG annotation paths around the anomalous regions. This visualization complements the quantitative ablation in Appendix Q.11: while the ablation measures how the number of injected layers affects detection metrics, Fig. 15 directly shows that multi-layer visual injection improves the decoder’s access to fine-grained glyph geometry without increasing the input sequence length.

Q.12. Generalizability of the Continuous Coordinate-Enhanced Decoder

To further validate that the proposed Continuous Coordinate-Enhanced Decoder (CCED) is a plug-and-play module that universally benefits vector graphic generation, we extended this ablation study to include all baseline models listed in Tab. 23. Specifically, we replaced the default discrete token-based coordinate prediction heads in these 9 representative models with our continuous regression branch, keeping all other training hyperparameters and data configurations identical.

As shown in Tab. 23, the integration of CCED yields consistent and significant improvements across all 9 baseline models, particularly in vector-level topological metrics (DTW, LDTW) and geometric precision (CD). These results strongly demonstrate that the quantization errors introduced by traditional discrete tokenization severely hinder the performance of large language models in high-precision geometric localization tasks. By effectively decoupling SVG structural semantic generation from continuous coordinate geometric regression, the proposed decoder design successfully overcomes this bottleneck. This comprehensive experiment convincingly proves that CCED is not a specialized design

rigidly tied to a specific architecture, but rather a core component with robust generalizability and broad applicability.

R. ANALYSIS

R.1. Sensitivity Analysis of the Number of Reference Templates

In the practical application of large-scale Chinese character font quality control, due to the vast Chinese character set, obtaining high-quality style reference templates that have been meticulously reviewed by experts is a challenging and costly task. Therefore, from the perspective of practical deployment, it would be of immense value if the model could achieve high-precision detection with the minimal number of reference templates. Based on this, we explored the specific impact of the number of reference templates K on detection accuracy, aiming to verify the model’s robustness under conditions of limited reference data.

As shown in Fig. 16, we analyzed the trend of model performance as K increases from 0 to 20. Although increasing the number of reference characters indeed introduces richer style cues, leading to a slight improvement in detection metrics, we observed that the performance gain shows a significant diminishing marginal effect. Notably, our method demonstrates outstanding few-shot efficiency, achieving high-precision defect localization even under the extreme condition of providing only a single reference template ($K = 1$). However, when $K > 6$, the performance gain flattens, showing a clear diminishing marginal effect. This finding suggests that FontDoctor is capable of effectively capturing core style features from very limited data, greatly reducing reliance on large-scale manually created reference sets, proving its feasibility in real-world scenarios where high-quality reference templates are difficult to obtain.

Table 23: Quantitative evaluation of the Continuous Coordinate-Enhanced Decoder (CCED) applied to all baseline models.

Method	MSE ↓	DTW ↓	LDTW ↓	CD ↓
StarVector-8B	0.068	3.1152	3.5913	0.8149
+ CCED	0.058	2.6540	3.0520	0.6750
DeepSeek-VL-7B	0.062	2.8883	3.3874	0.7316
+ CCED	0.052	2.4150	2.8540	0.5980
Qwen2.5-VL-3B	0.053	2.6256	3.1023	0.5977
+ CCED	0.044	2.1530	2.5140	0.4520
OmniSVG-3B	0.052	2.5988	3.0815	0.5788
+ CCED	0.043	2.1050	2.4850	0.4350
InternSVG-8B	0.049	2.4850	2.9510	0.5120
+ CCED	0.040	1.9850	2.3650	0.3850
SVGen-7B	0.047	2.4120	2.8650	0.4850
+ CCED	0.038	1.9845	2.3120	0.3654
Yi-VL-6B	0.045	2.3586	2.8068	0.4612
+ CCED	0.036	1.8950	2.2540	0.3450
LLaVA-V1.6-7B	0.035	1.9833	2.3978	0.3393
+ CCED	0.028	1.6420	1.9560	0.2485
Qwen3-VL-4B	0.020	1.3981	1.6186	0.1312
+ CCED	0.016	1.1520	1.3450	0.0985

R.2. Style Sensitivity Analysis

To thoroughly investigate whether the FontDoctor framework truly learns and applies the style-specific characteristics of reference templates in a few-shot paradigm, we designed a cross-style reference experiment. The aim of this experiment was to explore the model’s behavior when there is a significant style difference between the reference style image and the character to be detected. We selected a font with a style similar to that of the defective character (Input) as the reference style template to verify whether the model learns the style-specific structural and stroke priors from the style reference. The qualitative results in Fig. 17 clearly reveal the model’s behavioral pattern. When there is a notable style difference between the reference and the character being tested, FontDoctor exhibits a strong style-guided effect, using the specific stroke structure and connection topology of the reference style as prior constraints. This finding strongly supports that FontDoctor successfully learns and internalizes style-specific knowledge within the MLLM framework, with its defect detection mechanism essentially transforming into high-dimensional style comparison reasoning. The model does not judge based on the absolute quality of the character itself, but rather on the relative geometric and structural differences between it and the provided style template. Therefore, FontDoctor is able to suppress irrelevant style transfer

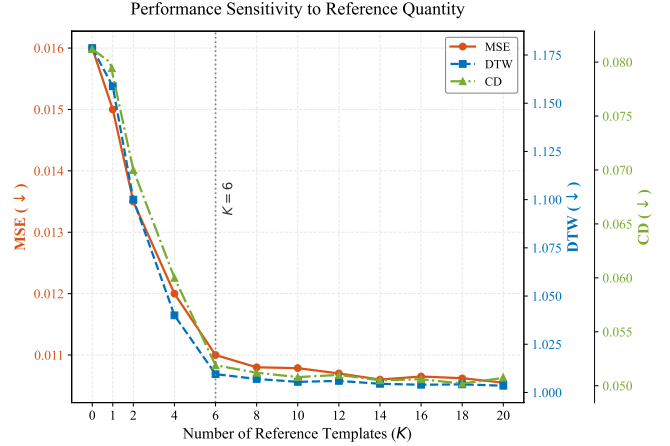


Fig. 16: The impact of the number of reference templates on detection performance. Despite the scarcity of high-quality reference data, the model maintains high accuracy with very few templates ($K = 1$). A saturation point is reached at $K = 6$, indicating that 6 templates are sufficient to establish a robust style prior.

under conditions of style mismatch, achieving accurate defect correction, and significantly enhancing the model’s practical utility and reliability in complex font quality control scenarios.

R.3. Verification of Reference Template Dependency

To verify whether FontDoctor strictly follows the given visual reference in a few-shot paradigm, rather than relying solely on the model’s internal generalized glyph priors, we designed an experiment to validate cross-style dependency. In this experiment, we deliberately manipulated the reference templates to create style conflicts. Specific strokes or components in the standard reference characters were replaced with distinctly different font styles, thus creating a mixed reference template with inconsistent style.

We used these altered templates as the standard reference to detect input characters that originally had a consistent style. The experimental results, shown in Fig. 18, demonstrate that although the input characters were completely correct in terms of general writing norms and topological structure, the model still sensitively detected the style mismatch with the reference template. It marked areas in the input character that did not include the reference template’s specific style features as defects. This result strongly supports the model’s ability to closely follow instructions and rely on visual cues, as it is able to perform comparative reasoning based strictly on the fine-grained style features of the reference template. This capability is crucial in font library design scenarios, as it ensures the model can adapt to any special style standards defined by the designer, without being interfered with by the generalized glyph knowledge from the model’s pretraining.

Content	挣	股	枕	烽	缕
Input	挣	股	枕	烽	缕
Reference	挠 铮	胖 没	枝 沈	烁 峰	绿 楼
Output	挣	股	枕	烽	缕

Fig. 17: Visualization results of cross-style defect detection. The model marks the style differences as defects, validating its reliance on the reference template style.

Content	骈	际	钞	粒	谅
Input	骈	际	钞	粒	谅
Reference	驱 拼	阳 标	铅 炒	粉 拉	识 凉
Output	骈	际	钞	粒	谅

Fig. 18: Visualization of the cross-style dependency validation experiment results. By artificially introducing style conflicts to construct mixed reference templates, the model marks input areas that conform to general norms but do not match the reference template’s style as defects.

R.4. Style Perception Under Unordered Stroke References

To further explore whether FontDoctor relies on global character semantics for template matching or has genuinely learned fine-grained local geometric style rules, we designed a stroke-level feature decomposition experiment. In the standard few-shot template reference paradigm, the model typically receives complete Chinese characters as references. However, in this experiment, we decomposed the reference character into independent strokes. As shown in Fig. 19, although the input reference template has been reduced to a collection of disorganized stroke fragments, FontDoctor still successfully identified and localized defects in the target character. This demonstrates that the model does not perform rigid, fixed template matching, nor does it rely on a complete global character structure prior. Instead, it is capable of extracting structural information from discrete visual cues and

transferring these local rules to the diagnosis of the target glyph.

R.5. Detection of Generative Artifacts

In current font generation tasks, whether based on GAN or Diffusion methods, the generated font images often contain stray pixel blocks. During the conversion process from raster images to vector graphics, these non-semantic pixel clusters are incorrectly parameterized as independent closed SVG paths, and these stray pixel blocks severely affect the usability of the font library.

To verify FontDoctor’s ability to detect such generative artifacts, we selected samples containing significant noise generated by a state-of-the-art font generation model [51] for testing. As shown in Fig. 20, these artifacts typically manifest as small closed regions that are detached from the main structure of the Chinese characters. The experimental results indicate

Content	唉	锐	媪	榜	地
Input	唉	锐	媪	榜	地
Style Reference	哎 挨	钛 说	好 温	板 滂	坎 他
Component Set	口 艾 扌 矣	钅 太 讠 兑	女 子 彡 显	木 反 彡 旁	土 欠 亻 也
Stroke					
Output	唉	锐	媪	榜	地

Fig. 19: Visualization of the stroke-level feature decomposition experiment. The original reference character is decomposed into independent stroke paths and randomly shuffled, thereby completely disrupting its global structural information.

that FontDoctor can perceive these isolated vector paths and accurately mark them as defects. The model is able to effectively distinguish between valid independent strokes and invalid generative noise, proving its potential for cleaning in the automated font generation post-processing pipeline.

R.6. Vector-Domain Repair of Generated Glyphs

Current few-shot font generation methods can rapidly synthesize target-style glyphs, but their outputs still often contain local contour jitter, broken strokes, abnormal adhesions, and stray noise. These defects may appear as subtle boundary artifacts in raster images, but after being converted to SVG by conventional vectorization tools, they can become redundant control points, incorrect closed paths, or discontinuous contours, affecting downstream font library production. To verify the role of FontDoctor in post-processing generated fonts, we use defective glyphs produced by few-shot font generation models as inputs and generate repaired SVG glyphs under the template-referenced setting.

Unlike traditional vectorization methods that passively trace pixel contours, FontDoctor formulates generated glyph optimization as a process that combines defect detection with structural repair. The model first localizes anomalous regions by jointly considering the query raster glyph, its SVG representation, and same-style reference templates, and then outputs repaired path structures in the vector domain. This allows FontDoctor to remove local artifacts introduced by generative models while preserving the original character structure and the target font style.

As shown in Fig. 21, the glyphs produced by generative models contain visible local structural anomalies, such as rough boundaries, stroke misalignment, and local deformation. FontDoctor identifies these regions that are inconsistent with the target style by referring to the templates and generates more coherent vector contours in the repaired results. This experiment shows that FontDoctor can not only de-

tect generative artifacts, but also repair them in the SVG space, providing an effective quality-improvement module before automatically generated fonts enter professional design workflows.

R.7. Geometric Correction across Diverse Font Styles

To further verify the defect repair ability of FontDoctor under complex font styles, we conduct a qualitative evaluation on glyphs with significant morphological variations, including calligraphic fonts, artistic fonts, and styles with special stroke connections. These glyphs vary substantially in stroke thickness, local curvature, connectivity, and decorative structures, making the boundary between legitimate stylistic variation and true geometric defects more ambiguous. This setting therefore poses a stronger challenge to the model’s style understanding and topological repair ability.

Unlike image-level repair methods that mainly pursue visual similarity, FontDoctor performs defect localization and path repair in the SVG space. It can explicitly modify abnormal contours, broken strokes, and incorrect connection regions while preserving the overall stylistic characteristics of the target font. As shown in Fig. 22, even for complex fonts with large morphological variations, FontDoctor generates repaired results with more coherent structures and more reasonable contours according to the reference templates.

Qualitative Comparison of Defect Repair. We further compare FontDoctor with existing methods on multi-style font repair to evaluate its detection–repair workflow in practical font production. As shown in Fig. 23, existing methods may introduce path intersections or remove local details during repair. FontDoctor instead generates topologically valid repair paths in the vector domain, producing contours that closely match the target glyphs while preserving the original font style.

This experiment shows that the repair ability of FontDoctor is not limited to regular printed fonts. For glyphs with drastic local curvature changes, complex stroke con-

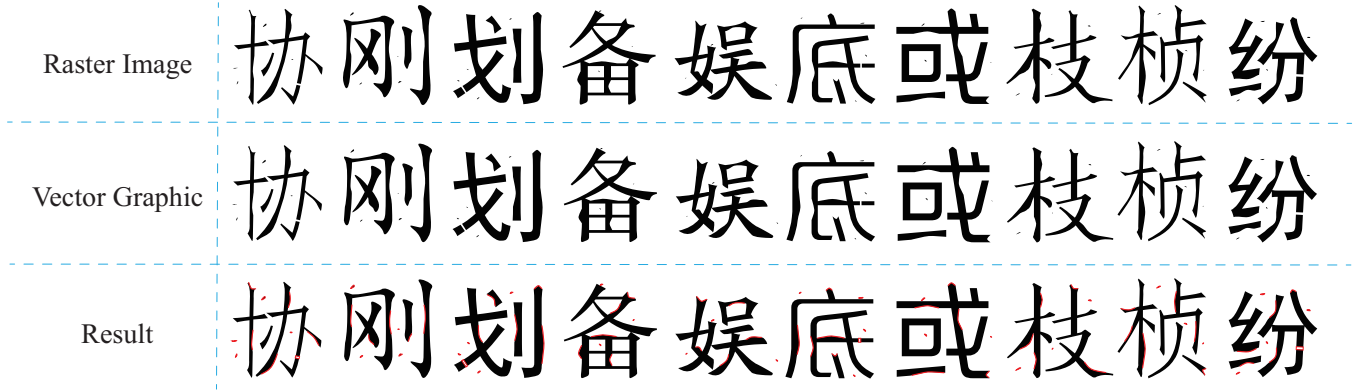


Fig. 20: Detection of artifacts produced by Chinese font generation models.

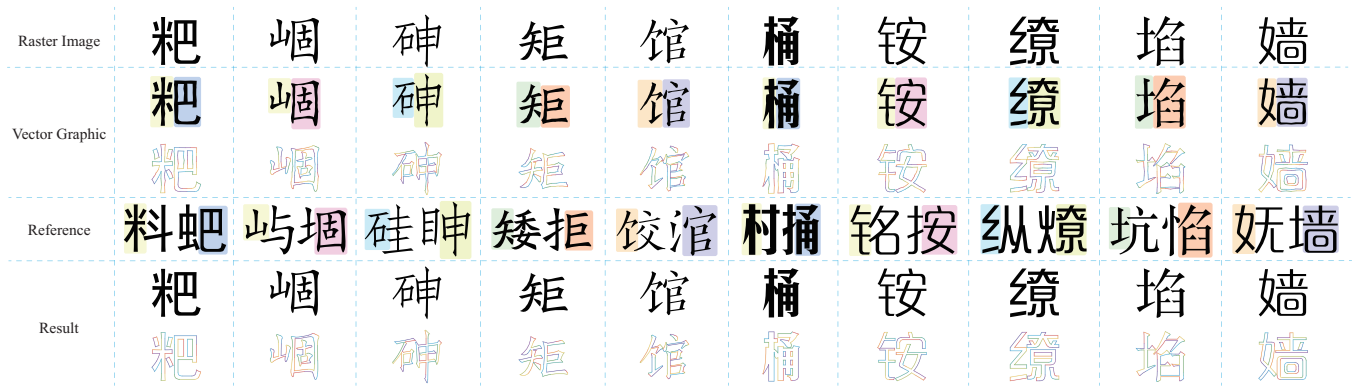


Fig. 21: Vector-domain defect repair for generated glyphs. FontDoctor repairs local defects in generated glyphs according to reference templates and outputs SVG glyphs with clearer structures and smoother contours.

nections, or strong decorative patterns, the model can still distinguish stylistic deformation from unintended defects and output editable repaired paths in the vector domain. This further demonstrates the practical value of FontDoctor as a defect detection and repair module in real font production workflows.

R.8. Quantitative Evaluation of Defect Repair

Table 24: Quantitative evaluation of defect repair.

Method	MSE ↓	DTW ↓	LDTW ↓	CD ↓	Accept. (%) ↑
StarVector-8B	0.042	2.1845	2.5062	0.4218	42.7
Qwen2.5-VL-3B	0.036	1.9520	2.2147	0.3356	51.3
Qwen3-VL-4B	0.024	1.4168	1.6354	0.1625	66.8
FontDoctor-4B (Ours)	0.009	0.8426	0.9314	0.0368	88.9
FontDoctor-8B (Ours)	0.008	0.8052	0.9025	0.0331	90.6

To quantify the closed-loop value of defect repair, we further evaluate the geometric consistency between the repaired SVG generated by each model and the designer-corrected ground truth. Specifically, we render the repaired SVG into raster images and compute MSE against the target repair result; meanwhile, we compute DTW, LDTW, and CD in

the vector domain. In addition, professional designers judge whether the repair result can be directly accepted or requires only minimal adjustment, yielding the designer acceptance rate (Accept.). The results are reported in Tab. 24.

The results show that general SVG or MLLM baselines can attempt to remove local anomalies, but they often damage the original stroke topology or introduce discontinuous paths, leading to higher vector errors and lower designer acceptance rates. In contrast, FontDoctor locates and repairs defect regions directly in the SVG structure, removing anomalies while preserving the original font style and path topology. As a result, it achieves the best performance on all geometric metrics and human acceptance.

R.9. Evaluation of SVG Optical Recognition Capability

To validate the effectiveness of the SVG optical compression strategy we proposed, we designed a dedicated experiment aimed at evaluating whether the visual encoder can function as an accurate optical reader. This experiment corresponds to Stage I of the training protocol, with the core objective being to verify whether the model can restore the visual embedding back to the original SVG XML text sequence by reading the rendered SVG code image. The experimental process is

Input	Output	Target	Input	Output	Target	Input	Output	Target	Input	Output	Target	Input	Output	Target
瑁	瑁	瑁	艙	艙	艙	私	私	私	括	括	括	嶙	嶙	嶙
泼	泼	泼	奋	奋	奋	梦	梦	梦	努	努	努	杞	杞	杞
请	请	请	柳	柳	柳	劬	劬	劬	匙	匙	匙	恹	恹	恹
湍	湍	湍	卸	卸	卸	晒	晒	晒	溢	溢	溢	僊	僊	僊
察	察	察	矿	矿	矿	玫	玫	玫	市	市	市	类	类	类
涪	涪	涪	甬	甬	甬	吾	吾	吾	消	消	消	倪	倪	倪
郎	郎	郎	纒	纒	纒	娛	娛	娛	蛛	蛛	蛛	盱	盱	盱
始	始	始	甬	甬	甬	钟	钟	钟	桩	桩	桩	侦	侦	侦

Fig. 22: Geometric defect repair across diverse font styles. FontDoctor repairs local topological defects in highly variable fonts such as calligraphic and artistic styles while preserving the target font style.

Table 25: Quantitative evaluation of SVG Optical Recognition.

Method	CER (%) ↓	BLEU-4 ↑	Syntax Validity (%) ↑	Exact Match (%) ↑
Tesseract OCR (v5)	6.25	0.88	82.40	18.50
PaddleOCR (v2)	3.12	0.94	91.50	42.30
Qwen2.5-VL-7B (Zero-shot)	0.18	0.98	99.20	93.50
GPT-4o (Zero-shot)	0.13	0.99	99.80	95.80
FontDoctor (Stage I)	0.11	0.99	99.90	96.50

shown in Fig. 24.

We randomly selected 5,000 samples from the CFDefect-2M test set and used the rendering function Φ to convert their SVG XML code into images with a resolution of $512 \times H$. The experimental task is defined as image-to-text generation. We selected two categories of baseline models for comparison: the traditional OCR engines, Tesseract-V5[63] and PaddleOCR-V2[64], which represent pure optical character recognition capabilities without understanding the semantics of the code; and Qwen2.5-VL[57] and GPT-4o, models with strong visual understanding and logical reasoning abilities, but not fine-tuned on specific code formatting data. The evaluation metrics include Character Error Rate (CER) to measure recognition accuracy at the micro level; BLEU-4 to evaluate the overall matching of long sequences; Syntax Validity to detect whether the generated code conforms to XML standards and can be parsed by the renderer; and the stringent Exact Match rate.

The results in Tab. 25 confirm that optical character recognition is a relatively simple task for structured code. Modern multimodal large models demonstrate impressive visual parsing capabilities. Both the open-source Qwen2.5-VL and the closed-source GPT-4o achieved CERs of less than 0.3%, with syntax validity close to 100%. This indicates that for current MLLMs, reading rendered SVG code presents almost no obstacles. FontDoctor achieved performance comparable to GPT-4o. This result proves that our strategy of converting SVG to visual embeddings Z_{svg} is equivalent in terms of information transmission to directly inputting text, but it significantly saves context window space, making it an efficient and near-lossless encoding method.

To intuitively and rigorously evaluate the effectiveness of the **SVG Optical Compression** strategy proposed in Sec. I, we conducted a re-rendering comparison experiment. The input in this experiment was not the original SVG text, but rather code text images simulating the view of a code editor, generated by **Algorithm 1**. The model needs to rely solely on the visual encoder to read this image and restore the highly compressed visual embedding Z_{svg} into a syntactically valid SVG XML sequence S_{pred} . To verify whether the geometric topology corresponding to the reconstructed code has been distorted, we rendered both the predicted sequence S_{pred} and the ground truth sequence S_{gt} into high-resolution glyph images using a standard rasterization engine, resulting in I_{glyph}^{pred} and I_{glyph}^{gt} . We then overlaid the two to generate a visual difference map. As shown in Fig. 25, the reconstructed re-

Input	羽	锄	琿	碲	绝	鲜
StarVector-8B	羽 羽	锄 锄	琿 琿	碲 碲	绝 绝	鲜 鲜
DeepSeek-VL-7B	羽 羽	锄 锄	琿 琿	碲 碲	绝 绝	鲜 鲜
Qwen2.5-VL-3B	羽 羽	锄 锄	琿 琿	碲 碲	绝 绝	鲜 鲜
OmniSVG-3B	羽 羽	锄 锄	琿 琿	碲 碲	绝 绝	鲜 鲜
InternSVG-8B	羽 羽	锄 锄	琿 琿	碲 碲	绝 绝	鲜 鲜
SVGen-7B	羽 羽	锄 锄	琿 琿	碲 碲	绝 绝	鲜 鲜
Yi-VL-6B	羽 羽	锄 锄	琿 琿	碲 碲	绝 绝	鲜 鲜
LLaVA-V1.6-7B	羽 羽	锄 锄	琿 琿	碲 碲	绝 绝	鲜 鲜
Qwen3-VL-4B	羽 羽	锄 锄	琿 琿	碲 碲	绝 绝	鲜 鲜
FontDoctor-4B(Ours)	羽 羽	锄 锄	琿 琿	碲 碲	绝 绝	鲜 鲜
FontDoctor-8B(Ours)	羽 羽	锄 锄	琿 琿	碲 碲	绝 绝	鲜 鲜
Target	羽 羽	锄 锄	琿 琿	碲 碲	绝 绝	鲜 鲜

Fig. 23: Qualitative comparison of defect repair across multiple font styles.

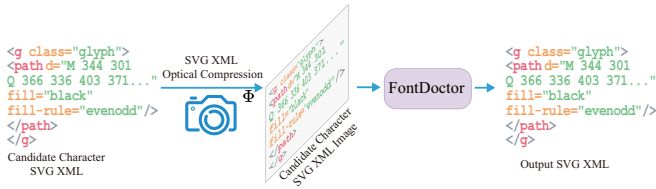


Fig. 24: Experimental process for validating the effectiveness of the SVG optical compression strategy.

sults achieved pixel-level exact alignment with the original input. This phenomenon indicates that FontDoctor successfully transformed the visual encoder into a high-precision optical reader, providing strong evidence for the effectiveness of the SVG optical compression strategy.

R.10. High-Fidelity Raster-to-Vector Conversion

Although the core task of FontDoctor is defect detection, Stage II: Raster-to-Vector Alignment in its training protocol

endows the model with precise image vectorization capabilities. In this stage, the model is explicitly trained to establish a semantic mapping between raster pixel distributions and SVG coordinate parameters by maximizing the conditional probability $p(S | \mathcal{E}_v(x_v))$. This explicit cross-modal alignment not only serves defect localization but also empowers the model with the potential to convert raster glyphs into high-quality parameterized vector graphics.

To validate this capability, as shown in Fig. 26, we input a set of standard defect-free raster Chinese character images into the model and instructed it to reconstruct their corresponding SVG XML code. The generated SVG code was then rendered into an image and compared by overlaying it with the original input. As shown in Fig. 27, the visualization results indicate that the Bézier curves generated by the model precisely align with the edges of the original raster strokes.

To further quantify FontDoctor’s vectorization performance, we compared it with the widely used traditional algorithm Potrace [65]. As shown in Tab. 26, the experimental results indicate that while Potrace performs reasonably well in fitting geometric contours, the paths it generates lack

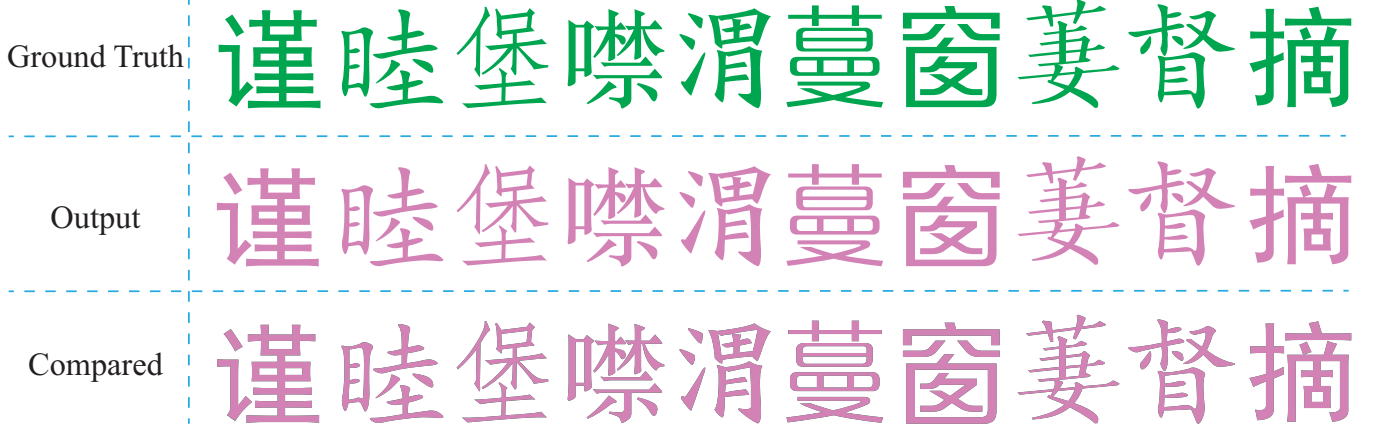


Fig. 25: Visual qualitative validation of SVG optical recognition and reconstruction capabilities.

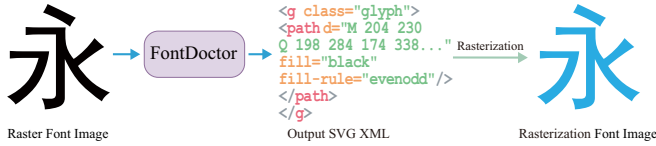


Fig. 26: Font image vectorization capability experimental process.

semantic order and have disorganized node distributions, resulting in poor performance on the DTW (Dynamic Time Warping) metric, which reflects the topological sequencing. In contrast, FontDoctor not only achieves the lowest reconstruction error on the CD metrics but also outperforms Potrace on the DTW and LDTW (Local DTW) metrics. This indicates that FontDoctor generates not only visually similar graphics but also SVG XML data aligned with the ground truth.

Table 26: Quantitative evaluation of Raster-to-Vector conversion performance.

Method	MSE ↓	DTW ↓	LDTW ↓	CD ↓
Potrace [65]	0.016	2.4508	2.8210	0.4612
FontDoctor (Ours)	0.019	1.1878	1.3414	0.1632

R.11. Glyph Image Vectorization

Table 27 presents a quantitative comparison of optimization-based and learning-based methods on the glyph image vectorization task. Optimization-based methods (e.g., Im2Vec and VecFontSDF) and early sequence generation models (e.g., DeepSVG and the DeepVecFont series) exhibit relatively higher errors in geometric and topological metrics (CD and DTW), primarily due to the lack of direct structural constraints given single-image inputs. VecFusion achieves strong performance by incorporating contour priors and pixel tex-

tures. However, FontDoctor, trained on the CFDefect-2M dataset, demonstrates superior comprehensive performance. Specifically, FontDoctor achieves the highest IoU (0.928) and the lowest Mean Squared Error (0.019), Chamfer Distance (0.1632), and Dynamic Time Warping (1.1878). These results indicate that FontDoctor can effectively reconstruct geometrically precise and topologically ordered vector paths directly from raster glyph images.

Table 27: Quantitative comparison of image-to-vector glyph reconstruction methods. The table evaluates optimization-based and learning-based approaches across visual (IoU, MSE) and topological (CD, DTW) metrics.

Methods	Vec. Superv.	Img-to-vec	IoU ↑	MSE ↓	CD ↓	DTW ↓
Im2Vec [20]	×	✓	0.768	0.103	0.5486	5.4520
VecFontSDF [66]	×	✓	0.782	0.092	0.5017	5.1240
DeepSVG [24]	✓	×	0.725	0.114	0.6128	5.8420
DeepVecFont [67]	✓	✓	0.812	0.064	0.4215	4.5620
DeepVecFont-v2 [68]	✓	✓	0.885	0.052	0.3126	3.8520
VecFusion [69]	✓	✓	0.914	0.028	0.2147	1.6945
FontDoctor (Ours)	✓	✓	0.928	0.019	0.1632	1.1878

R.12. Failure Case Analysis

Despite FontDoctor’s excellent performance in most standard print and handwritten fonts, it still faces certain challenges when handling special fonts with highly artistic features, particularly calligraphy fonts. As shown in Fig. 28, the main failure modes can be summarized as follows:

Challenges in Calligraphy Fonts: Calligraphy fonts often simulate the natural texture of brush strokes, pursuing artistic effects such as “flying white” and “dry strokes.” This results in character contours naturally having irregular jagged edges and discontinuous topological structures. Specifically, the legitimate artistic rough edges in calligraphy fonts are often misinterpreted by the model as “smoothness defects” or “edge noise.” Additionally, within the same stroke, calligraphy fonts frequently exhibit drastic changes in thickness and

Raster Image	探亮念渡乘媒举丝临峤
Vector Graphic	探亮念渡乘媒举丝临峤
Compared	探亮念渡乘媒举丝临峤

Fig. 27: Visualization of Raster-to-Vector conversion results. The output characters align perfectly with the ground truth.

Content	帆 响 嫖 姬 操 风	屋 推 怙 愿 榻 忽
Input	帆 响 嫖 姬 操 风	屋 推 怙 愿 榻 忽
Output	帆 响 嫖 姬 操 风	屋 推 怙 愿 榻 忽
Target	帆 响 嫖 姬 操 风	屋 推 怙 愿 榻 忽

Fig. 28: Failure cases validation. (a) The natural effect in calligraphic fonts is misinterpreted as contour defects. (b) Highly decorative fonts lead to structural recognition ambiguity.

morphological distortion. This high variability makes it difficult for the model to accurately define the boundary between “design intent” and “structural imbalance” in the absence of clear reference templates, leading to missed or misclassified structural defects.

Ambiguity in Highly Decorative Fonts: For some highly exaggerated decorative fonts, the stroke structure deviates significantly from the standard skeleton of Chinese characters, disrupting the character’s connectivity features. In such cases, the SVG optical compression strategy may lose small decorative metadata that constitute the glyph, causing the model to incorrectly interpret the overall semantic structure of the character and make erroneous inferences about the stroke connection relationships.

These limitations suggest that future work needs to incorporate prior knowledge of specific artistic styles or develop more adaptable fine-grained texture descriptors to balance normative detection with artistic inclusiveness.

R.13. Defect Detection on Multi-Style Fonts

The previous experiments have validated the effectiveness of the proposed method in standard defect detection scenarios. This section further demonstrates the model’s performance in detecting defects across a more diverse set of font styles to assess its robustness against style variations. As shown in Fig. 29, we conducted defect detection experiments on a collection of various fonts, including standard Songti, Heiti, as well as complex calligraphy and artistic styles. By comparing the results, it is evident that despite significant differences in stroke thickness, connection methods, and topological structures, the proposed method still achieves consistent and precise defect localization. This strongly proves the model’s excellent generalization ability, enabling it to effectively adapt to the inherent high variability of Chinese character font styles.

R.14. Cross-Language Defect Detection

We extended the defect detection experiment to English, Japanese, and Korean. The experimental results, as shown in Fig. 30, demonstrate that although the model was not trained



Fig. 29: Qualitative results of defect detection on multi-style fonts. The model consistently identifies and localizes defects across various styles, including Songti, Heiti, and artistic fonts, demonstrating strong generalization ability.



Fig. 30: Cross-language defect detection.

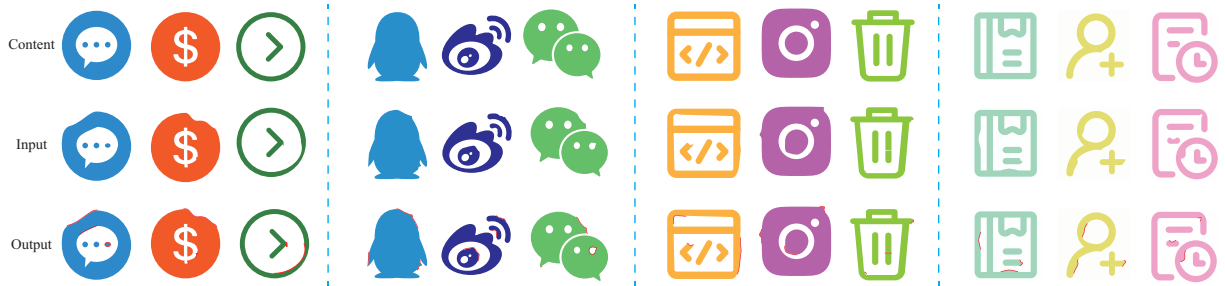


Fig. 31: Vector icon defect detection.

on datasets from these languages, it still exhibits excellent generalization performance. This strong generalization capability provides powerful technical support for cross-lingual font design and its downstream applications.

R.15. Vector Icon Defect Detection

We extend the experiment to vector icons with more complex structures. As shown in Fig. 31, despite the significant topological differences between icons and text, the model can still

accurately locate defects, demonstrating its potential for handling vector graphic tasks.

R.16. Diagnostic Comparison with Closed-Source MLLMs

To evaluate whether general closed-source multimodal models can directly handle SVG font defect localization without task-specific fine-tuning, we conduct a diagnostic zero-shot comparison. This experiment is not used as a leaderboard of closed-source models; rather, it examines the task

Table 28: Diagnostic zero-shot comparison with representative closed-source general MLLMs. Exact API model identifiers and invalid-output rates are reported to make the snapshot auditable.

Provider	API Model ID	Invalid (%) ↓	IoU ↑	MSE ↓	DTW ↓	LDTW ↓	CD ↓
Baidu	ernie-5.0	86.4	0.045	0.925	312.450	345.600	5.8420
xAI	Grok-4.5	82.8	0.052	0.894	285.300	318.550	5.2100
Z.ai	GLM-5.2	79.6	0.061	0.862	264.150	295.400	4.8550
Anthropic	Claude-Sonnet-4.6	74.2	0.085	0.815	242.800	274.150	4.5200
Google	Gemini-3.6-Flash	69.8	0.092	0.785	215.500	248.600	4.1050
Moonshot AI	Kimi-K3	66.6	0.098	0.768	198.250	225.300	3.8500
OpenAI	GPT-5.6-Sol	60.4	0.115	0.725	175.400	198.850	3.4250
Anthropic	Claude-Opus-5	57.2	0.124	0.710	162.800	185.500	3.1200
Ours	FontDoctor-4B	0.0	0.925	0.016	1.1786	1.2825	0.0812

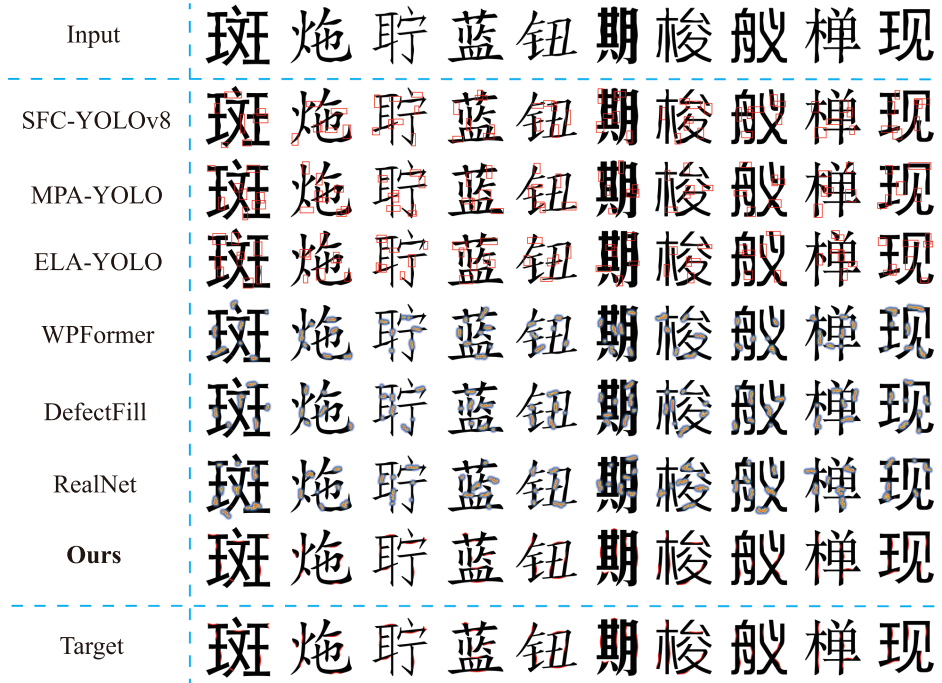


Fig. 32: Qualitative comparison between FontDoctor and traditional image-based industrial anomaly detection methods. Image-based methods only provide coarse-grained pixel-level regional indications, whereas FontDoctor outputs precise topological localization and vector path corrections.

adaptability of general MLLMs when task-specific training, SVG optical compression, and continuous-coordinate decoding are absent. Since closed-source APIs are continuously updated, we treat this experiment as an auditable time-stamped snapshot and record the provider, API model identifier, query date, decoding parameters, input resolution, unified prompt, and raw output. All closed-source models are evaluated on the same 500-sample diagnostic subset using a single-turn query, without model-specific prompt tuning, retrying failed responses, or manually repairing outputs. The subset is stratified from the official test set by real/synthetic defects, font style, and stroke-count bins. We set temperature to 0 and top- p to 1. Each input contains the query glyph image and its corresponding rendered SVG optical-compression image,

and the prompt requires the model to output only parsable SVG defect annotation paths.

Unified prompt. *Given the glyph image and the rendered SVG-XML image, locate all font defects. Return only a valid SVG fragment containing red defect annotation paths. Do not output explanations, Markdown, or natural language. If no defect exists, return an empty SVG fragment.*

For evaluation, we first check whether each output is a syntactically valid and renderable SVG defect annotation using the same SVG parser and renderer. An output is counted as invalid if it cannot be parsed, cannot be rendered, lacks defect paths, or contains task-irrelevant natural language instead of SVG annotations. For invalid outputs, IoU is set to 0; MSE is computed between a blank prediction map and the

ground-truth defect map; DTW, LDTW, and CD are assigned the maximum penalty values within this diagnostic subset. All failed samples are retained when computing the averages. Because the serving versions of closed-source models may change over time, the results in Tab. 28 should be interpreted as a fixed-date diagnostic reference rather than a fully stable public leaderboard. The results indicate that general closed-source models can recognize some visual anomalies, but remain unstable when required to output strictly renderable SVG defect paths with precise coordinates.

R.17. Qualitative Comparison with Image-Based Industrial Detection Methods

Existing defect detection methods have been widely applied in industrial manufacturing, but most rely on pure image features for pixel-level segmentation or coarse-grained regression of anomalous regions. To highlight the advantages of the SVG-based intermediate modality, we qualitatively compared FontDoctor with several mainstream image-based Industrial Anomaly Detection (IAD) methods, specifically including SFC-YOLOv8 [70], MPA-YOLO [71], ELA-YOLO [72], WPFormer [73], DefectFill [74], and RealNet [75]. As shown in Fig. 32, although traditional image-based methods can roughly perceive anomalous areas in font images, their outputs are limited to coarse-grained heatmaps or irregular pixel masks. Because Chinese font defects are fundamentally misalignments of Bézier curves and topological structures, pure pixel-level masks cannot provide clear geometric guidance on “how to repair”. In contrast, FontDoctor operates directly in the vector domain, generating defect annotations as SVG paths that precisely fit the anomalous contours. This not only achieves ultra-high precision localization at the control point level but also provides directly usable structured data support for subsequent automated repairs and designer reviews, completely breaking through the application bottlenecks of traditional computer vision methods in font design workflows.

S. REFERENCES

- [1] Xinyu Liu, Yao Li, Yiyu Guo, and Luoyu Zhou, “Print-ing defect detection based on scale-adaptive template matching and image alignment,” *Sensors*, vol. 23, no. 9, pp. 4414, 2023.
- [2] Swarit Anand Singh and Kaushal A Desai, “Automated surface defect detection framework using machine vision and convolutional neural networks,” *Journal of Intelligent Manufacturing*, vol. 34, no. 4, pp. 1995–2011, 2023.
- [3] Ross Girshick, Jeff Donahue, Trevor Darrell, and Jiten-dra Malik, “Rich feature hierarchies for accurate object detection and semantic segmentation,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2014, pp. 580–587.
- [4] Zirou Jiang, Jintao Fu, Tianchen Zeng, Renjie Liu, Peng Cong, Jichen Miao, and Yuewen Sun, “Defect r-cnn: A novel high-precision method for ct image defect detec-tion,” *Applied Sciences*, vol. 15, no. 9, pp. 4825, 2025.
- [5] Jonathan Long, Evan Shelhamer, and Trevor Darrell, “Fully convolutional networks for semantic segmenta-tion,” in *Proceedings of the IEEE conference on com-puter vision and pattern recognition*, 2015, pp. 3431–3440.
- [6] Meixia Fu, Jiansheng Wu, Qu Wang, Lei Sun, Zhangchao Ma, Chaoyi Zhang, Wanqing Guan, Wei Li, Na Chen, Danshi Wang, et al., “Region-based fully con-volutional networks with deformable convolution and attention fusion for steel surface defect detection in in-dustrial internet of things,” *IET Signal Processing*, vol. 17, no. 5, pp. e12208, 2023.
- [7] Gongjie Zhang, Kaiwen Cui, Tzu-Yi Hung, and Shijian Lu, “Defect-gan: High-fidelity defect synthesis for au-tomated defect inspection,” in *2021 IEEE winter confer-ence on applications of computer vision (WACV)*. IEEE, 2021, pp. 2523–2533.
- [8] Shuanlong Niu, Bin Li, Xinggang Wang, and Hui Lin, “Defect image sample generation with gan for improv-ing defect recognition,” *IEEE Transactions on Automa-tion Science and Engineering*, vol. 17, no. 3, pp. 1611–1622, 2020.
- [9] Junpu Wang, Guili Xu, Fujun Yan, Jinjin Wang, and Zhengsheng Wang, “Defect transformer: An efficient hybrid transformer architecture for surface defect detec-tion,” *Measurement*, vol. 211, pp. 112614, 2023.
- [10] Asim Niaz, Muhammad Umraiz, Shafiullah Soomro, and Kwang Nam Choi, “Vision transformer and mamba-attention fusion for high-precision pcb defect detection,” *PloS one*, vol. 20, no. 9, pp. e0331175, 2025.
- [11] James Richard Diebel, *Bayesian Image Vectorization: the probabilistic inversion of vector image rasterization*, Stanford University, 2008.
- [12] Zicheng Liao, Hugues Hoppe, David Forsyth, and Yizhou Yu, “A subdivision-based representation for vec-tor image editing,” *IEEE transactions on visualization and computer graphics*, vol. 18, no. 11, pp. 1858–1867, 2012.
- [13] Tian Xia, Binbin Liao, and Yizhou Yu, “Patch-based image vectorization with automatic curvilinear feature alignment,” *ACM Transactions on Graphics (TOG)*, vol. 28, no. 5, pp. 1–10, 2009.
- [14] Tzu-Mao Li, Michal Lukáč, Michaël Gharbi, and Jonathan Ragan-Kelley, “Differentiable vector graphics rasterization for editing and learning,” *ACM Transac-tions on Graphics (TOG)*, vol. 39, no. 6, pp. 1–15, 2020.
- [15] Tetiana Marusenkova, “A potrace-based algorithm for extracting handwriting line spacing, baseline, and slant,” in *2024 IEEE 19th International Conference on Com-puter Science and Information Technologies (CSIT)*. IEEE, 2024, pp. 1–4.
- [16] Yiren Song, Danze Chen, and Mike Zheng Shou, “Lay-ertracer: Cognitive-aligned layered svg synthesis via diffusion transformer,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025, pp. 19731–19741.
- [17] David Ha and Douglas Eck, “A neural representation of sketch drawings,” *arXiv preprint arXiv:1704.03477*, 2017.
- [18] I-Chao Shen and Bing-Yu Chen, “Clipgen: A deep generative model for clipart vectorization and synthe-sis,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 28, no. 12, pp. 4211–4224, 2021.
- [19] Yiren Song, Xuning Shao, Kang Chen, Weidong Zhang, Zhongliang Jing, and Minzhe Li, “Clipvg: Text-guided image manipulation using differentiable vector graph-ics,” in *Proceedings of the AAAI conference on artificial intelligence*, 2023, vol. 37, pp. 2312–2320.
- [20] Pradyumna Reddy, Michael Gharbi, Michal Lukac, and Niloy J Mitra, “Im2vec: Synthesizing vector graph-ics without vector supervision,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 7342–7351.
- [21] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al., “Language models are unsupervised multitask learners,” *OpenAI blog*, vol. 1, no. 8, pp. 9, 2019.
- [22] Diederik P Kingma and Max Welling, “Auto-encoding variational bayes,” *arXiv preprint arXiv:1312.6114*, 2013.
- [23] Defu Cao, Zhaowen Wang, Jose Echevarria, and Yan Liu, “Svgformer: Representation learning for continu-ous vector graphics using transformers,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 10093–10102.
- [24] Alexandre Carlier, Martin Danelljan, Alexandre Alahi, and Radu Timofte, “Deepsvg: A hierarchical genera-tive network for vector graphics animation,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 16351–16361, 2020.
- [25] Ronghuan Wu, Wanchao Su, Kede Ma, and Jing Liao, “Iconshop: Text-guided vector icon synthesis with

- autoregressive transformers,” *ACM Transactions on Graphics (TOG)*, vol. 42, no. 6, pp. 1–14, 2023.
- [26] Zehao Chen and Rong Pan, “Svgbuilder: Component-based colored svg generation with text-guided autoregressive transformers,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025, vol. 39, pp. 2358–2366.
- [27] Haruka Aoki and Kiyoharu Aizawa, “Svg vector font generation for chinese characters with transformer,” in *2022 IEEE international conference on image processing (ICIP)*. IEEE, 2022, pp. 646–650.
- [28] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
- [29] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al., “Language models are few-shot learners,” *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [30] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al., “Gpt-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [31] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al., “Llama: Open and efficient foundation language models,” *arXiv preprint arXiv:2302.13971*, 2023.
- [32] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al., “Improving language understanding by generative pre-training,” 2018.
- [33] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt, “Codesearch-net challenge: Evaluating the state of semantic code search,” *arXiv preprint arXiv:1909.09436*, 2019.
- [34] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, et al., “The pile: An 800gb dataset of diverse text for language modeling,” *arXiv preprint arXiv:2101.00027*, 2020.
- [35] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al., “Starcoder: may the source be with you!,” *arXiv preprint arXiv:2305.06161*, 2023.
- [36] Ekaba Bisong et al., *Building machine learning and deep learning models on Google cloud platform*, Springer, 2019.
- [37] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al., “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [38] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong, “Codegen: An open large language model for code with multi-turn program synthesis,” *arXiv preprint arXiv:2203.13474*, 2022.
- [39] Xi Jiang, Jian Li, Hanqiu Deng, Yong Liu, Bin-Bin Gao, Yifeng Zhou, Jialin Li, Chengjie Wang, and Feng Zheng, “Mmad: A comprehensive benchmark for multimodal large language models in industrial anomaly detection,” in *International conference on learning representations*, 2025, vol. 2025, pp. 87273–87295.
- [40] Juan A Rodriguez, Abhay Puri, Shubham Agarwal, Issam H Laradji, Pau Rodriguez, Sai Rajeswar, David Vazquez, Christopher Pal, and Marco Pedersoli, “Starvector: Generating scalable vector graphics code from images and text,” in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 16175–16186.
- [41] Ximing Xing, Juncheng Hu, Guotao Liang, Jing Zhang, Dong Xu, and Qian Yu, “Empowering llms to understand and generate complex vector graphics,” in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 19487–19497.
- [42] Siqi Chen, Xinyu Dong, Haolei Xu, Xingyu Wu, Fei Tang, Hang Zhang, Yuchen Yan, Linjuan Wu, Wenqi Zhang, Guiyang Hou, et al., “Svgenius: Benchmarking llms in svg understanding, editing and generation,” in *Proceedings of the 33rd ACM International Conference on Multimedia*, 2025, pp. 13289–13296.
- [43] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al., “Gpt-4o system card,” *arXiv preprint arXiv:2410.21276*, 2024.
- [44] Qihao Zhu, Daya Guo, Zhihong Shao, Dejian Yang, Peiyi Wang, Runxin Xu, Y Wu, Yukun Li, Huazuo Gao, Shirong Ma, et al., “Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence,” *arXiv preprint arXiv:2406.11931*, 2024.
- [45] Yuxin Kong, Canjie Luo, Weihong Ma, Qiyuan Zhu, Shenggao Zhu, Nicholas Yuan, and Lianwen Jin, “Look closer to supervise better: One-shot font generation via component-based discriminator,” in *2022 IEEE/CVF conference on computer vision and pattern recognition (CVPR)*. IEEE, 2022, pp. 13472–13481.
- [46] Licheng Tang, Yiyang Cai, Jiaming Liu, Zhibin Hong, Mingming Gong, Minhu Fan, Junyu Han, Jingtuo Liu, Errui Ding, and Jingdong Wang, “Few-shot font generation by learning fine-grained local styles,” in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2022, pp. 7885–7894.
- [47] Chi Wang, Min Zhou, Tiezheng Ge, Yuning Jiang, Hujun Bao, and Weiwei Xu, “Cf-font: Content fusion for few-shot font generation,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 1858–1867.

- [48] Haibin He, Xinyuan Chen, Chaoyue Wang, Juhua Liu, Bo Du, Dacheng Tao, and Qiao Yu, "Diff-font: Diffusion model for robust one-shot font generation," *International Journal of Computer Vision*, vol. 132, no. 11, pp. 5372–5386, 2024.
- [49] Zhenhua Yang, Dezhi Peng, Yuxin Kong, Yuyi Zhang, Cong Yao, and Lianwen Jin, "Fontdiffuser: One-shot font generation via denoising diffusion with multi-scale content aggregation and style contrastive learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024, vol. 38, pp. 6603–6611.
- [50] Bin Fu, Fanghua Yu, Anran Liu, Zixuan Wang, Jie Wen, Junjun He, and Yu Qiao, "Generate like experts: Multi-stage font generation by incorporating font transfer process into diffusion models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 6892–6901.
- [51] Yu Liu, Yang Ding, Fatimah Binti Khalid, Cunrui Wang, and Lei Wang, "Few-shot font generation via denoising diffusion and component-level fine-grained style," *Expert Systems with Applications*, vol. 296, pp. 128987, 2026.
- [52] Haoran Wei, Yaofeng Sun, and Yukun Li, "Deepseek-ocr: Contexts optical compression," *arXiv preprint arXiv:2510.18234*, 2025.
- [53] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee, "Visual instruction tuning," *Advances in neural information processing systems*, vol. 36, pp. 34892–34916, 2023.
- [54] Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee, "Improved baselines with visual instruction tuning," in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2024, pp. 26286–26296.
- [55] Alex Young, Bei Chen, Chao Li, Chengen Huang, Ge Zhang, Guanwei Zhang, Guoyin Wang, Heng Li, Jiangcheng Zhu, Jianqun Chen, et al., "Yi: Open foundation models by 01. ai," *arXiv preprint arXiv:2403.04652*, 2024.
- [56] Haoyu Lu, Wen Liu, Bo Zhang, Bingxuan Wang, Kai Dong, Bo Liu, Jingxiang Sun, Tongzheng Ren, Zhuoshu Li, Hao Yang, et al., "Deepseek-vl: towards real-world vision-language understanding," *arXiv preprint arXiv:2403.05525*, 2024.
- [57] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibao Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al., "Qwen2. 5-vl technical report," *arXiv preprint arXiv:2502.13923*, 2025.
- [58] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, et al., "Qwen3-vl technical report," *arXiv preprint arXiv:2511.21631*, 2025.
- [59] Zhou Wang, Alan C Bovik, Hamid R Sheikh, and Eero P Simoncelli, "Image quality assessment: from error visibility to structural similarity," *IEEE transactions on image processing*, vol. 13, no. 4, pp. 600–612, 2004.
- [60] Hiroaki Sakoe and Seibi Chiba, "Dynamic programming algorithm optimization for spoken word recognition," *IEEE transactions on acoustics, speech, and signal processing*, vol. 26, no. 1, pp. 43–49, 2003.
- [61] Yunyue Zhu and Dennis Shasha, "Warping indexes with envelope transforms for query by humming," in *Proceedings of the 2003 ACM SIGMOD international conference on Management of data*, 2003, pp. 181–192.
- [62] Haoqiang Fan, Hao Su, and Leonidas J Guibas, "A point set generation network for 3d object reconstruction from a single image," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 605–613.
- [63] Tesseract OCR Contributors, "Tesseract ocr: Open source optical character recognition engine," <https://github.com/tesseract-ocr/tesseract>, 2024, Accessed: 2025-12-20.
- [64] Yuning Du, Chenxia Li, Ruoyu Guo, Cheng Cui, Weiwei Liu, Jun Zhou, Bin Lu, Yehua Yang, Qiwen Liu, Xiaoguang Hu, et al., "Pp-ocrv2: Bag of tricks for ultra lightweight ocr system," *arXiv preprint arXiv:2109.03144*, 2021.
- [65] Peter Selinger, "Potrace: a polygon-based tracing algorithm," 2003.
- [66] Zeqing Xia, Bojun Xiong, and Zhouhui Lian, "Vec-fontsdf: Learning to reconstruct and synthesize high-quality vector fonts via signed distance functions," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2023, pp. 1848–1857.
- [67] Yizhi Wang and Zhouhui Lian, "Deepvecfont: synthesizing high-quality vector fonts via dual-modality learning," *ACM Transactions on Graphics (TOG)*, vol. 40, no. 6, pp. 1–15, 2021.
- [68] Yuqing Wang, Yizhi Wang, Longhui Yu, Yuesheng Zhu, and Zhouhui Lian, "Deepvecfont-v2: Exploiting transformers to synthesize vector fonts with higher quality," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2023, pp. 18320–18328.
- [69] Vikas Thamizharasan, Difan Liu, Shantanu Agarwal, Matthew Fisher, Michaël Gharbi, Oliver Wang, Alec Jacobson, and Evangelos Kalogerakis, "Vecfusion: Vector font generation with diffusion," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 7943–7952.
- [70] Huan Liu, Ruikuan Hu, Haobin Dong, and Zheng Liu, "Sfc-yolov8: Enhanced strip steel surface defect detection using spatial-frequency domain-optimized yolov8," *IEEE Transactions on Instrumentation and Measurement*, 2025.
- [71] Yanli Zhou and Zhanfang Zhao, "Mpa-yolo: Steel surface defect detection based on improved yolov8 framework," *Pattern Recognition*, vol. 168, pp. 111897, 2025.
- [72] Ruichen Ma, Jinglong Chen, Yong Feng, Zitong Zhou, and Jingsong Xie, "Ela-yolo: An efficient method with

linear attention for steel surface defect detection during manufacturing,” *Advanced Engineering Informatics*, vol. 65, pp. 103377, 2025.

- [73] Feng Yan, Xiaoheng Jiang, Yang Lu, Jiale Cao, Dong Chen, and Mingliang Xu, “Wavelet and prototype augmented query-based transformer for pixel-level surface defect detection,” in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 23860–23869.
- [74] Jaewoo Song, Daemin Park, Kanghyun Baek, Sangyub Lee, Jooyoung Choi, Eunji Kim, and Sungroh Yoon, “Defectfill: Realistic defect generation with inpainting diffusion model for visual inspection,” in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 18718–18727.
- [75] Ximiao Zhang, Min Xu, and Xiuzhuang Zhou, “Realnet: A feature selection network with realistic synthetic anomaly for anomaly detection,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2024, pp. 16699–16708.